

GRAPH-BASED KV CACHE COMPRESSION WITH ANSWER SUPERVISION

Anonymous authors

Paper under double-blind review

ABSTRACT

Language-model agents perform multistep tasks by accumulating conversation histories, retrieved documents, and tool outputs. As these contexts grow, the key-value (KV) cache becomes a substantial memory bottleneck. Compression is particularly challenging in these settings: information introduced early may support later decisions, and relevant evidence may be distributed across multiple observations. These demands motivate cache eviction policies that consider relationships among tokens and their utility for subsequent generation. We introduce a method that uses graph neural networks (GNNs) to guide KV cache compression. Building on FastKVzip, our selector augments token representations through message passing before predicting retention scores. Training proceeds in two stages. First, a warm-up stage uses importance scores computed by KVzip’s context reconstruction procedure as supervision, providing an informed starting point for cache selection. Second, answer-supervised fine-tuning aligns the selector with the underlying language model’s generation behavior: the selector learns to retain information that allows the model to reproduce answers generated with its full cache. The language model remains frozen throughout, while the selector is trained across varying compression budgets. Our approach thus connects relational reasoning over context with supervision from the model’s own answers, targeting the preservation of task-relevant information within a limited cache.

1 INTRODUCTION

Language-model agents build solutions over a sequence of interactions: they retrieve evidence, invoke tools, inspect observations, and revise their next actions. Frameworks such as ReAct make this interaction between reasoning and environmental feedback explicit (Yao et al., 2023), while memory-management systems address the difficulty of maintaining useful information over extended sessions (Packer et al., 2023). The accumulated history is therefore part of the agent’s working context. Keeping this context available incurs a growing cost: autoregressive Transformers store key-value (KV) representations of previous tokens to avoid recomputing them during decoding, and this cache grows with the sequence length. Efficient cache management is consequently central to serving long interactions under a fixed memory budget (Kwon et al., 2023).

The compression problem is not determined by context length alone. A tool output may contain a fact needed several actions later, and an answer may depend on evidence spread across different observations. Recency and relevance to the current question can therefore provide incomplete guidance about what to retain. Query-agnostic compression addresses this setting by constructing a compressed cache before a particular downstream question is known (Kim et al., 2025). The objective is to preserve information that the language model can use across subsequent requests, while removing cache entries whose contribution does not justify their memory cost.

KVzip estimates this contribution by asking the underlying language model to reconstruct the context and measuring how strongly cached entries are attended to during reconstruction (Kim et al., 2025). FastKVzip learns a lightweight gate to predict these scores from hidden representations, avoiding reconstruction when scoring a new context (Kim et al., 2026). Its gate operates separately on each token representation. These representations already contain context from the frozen Transformer, but the selector itself does not learn an additional exchange of information across token

positions. This motivates examining whether explicit relational modeling within the selector can better support cache retention decisions.

We develop a graph-based cache selector that augments Fast KVzip with learned message passing within context blocks. Tokens form the nodes of a graph whose edge weights depend on their hidden representations. Separate graph parameters for each layer and KV head allow the selector to represent different relationships within the same context. The resulting messages enrich the representations used to predict retention scores; the retained KV vectors themselves remain unchanged. Graph-based eviction has also been explored through similarity-based score propagation (Li et al., 2025a). Our focus is on learning the relational scoring function and adapting its decisions to the behavior of the frozen language model.

Training has two stages with distinct roles. Reconstruction-score supervision first serves as a warm-up, providing direct targets for individual cache entries and an informed starting point for the selector. Answer supervision then adapts this selector to downstream generation: given a context and question, the full-cache model supplies a reference answer, and the selector is trained so that the same model can predict that answer from a compressed cache. The question supplies training feedback without becoming an input to the context selector. This separates a broadly available importance signal from an objective tied to how the model uses the retained information.

[Final introduction paragraph reserved for the principal empirical findings and the paper’s main contributions.]

2 RELATED WORK

2.1 ATTENTION MECHANISM AND KV CACHING

Consider a context of T tokens and a Transformer layer ℓ with hidden states $X^{(\ell)} \in \mathbb{R}^{T \times d}$. In multi-head attention, head h at this layer uses learned projections to form query, key, and value matrices (Vaswani et al., 2017):

$$Q_h = X^{(\ell)} W_{Q,h}, \quad K_h = X^{(\ell)} W_{K,h}, \quad V_h = X^{(\ell)} W_{V,h}. \quad (1)$$

Each projection maps the hidden states to a head dimension d_h ; normalization and positional transformations are omitted here for clarity.

During autoregressive decoding, the current token’s query $q_{T,h}$, the T -th row of Q_h , computes the head output

$$o_{T,h} = \text{softmax}\left(\frac{q_{T,h} K_h^\top}{\sqrt{d_h}}\right) V_h, \quad (2)$$

where K_h and V_h contain the keys and values up to position T , including the current token. Causal attention makes earlier representations depend only on their prefixes, so appending a token leaves their keys and values unchanged. These can therefore be reused from the KV cache, to which only the current key and value are appended. Earlier queries need not be retained: each decoding step uses its own query to read the stored keys and values. Grouped-query attention (GQA) further reduces cache storage by sharing key and value projections among groups of query heads (Ainslie et al., 2023).

Attention heads can serve different functions (Voita et al., 2019). For example, DuoAttention identifies retrieval heads that need access to distant tokens and streaming heads that can rely on recent tokens and a few initial attention sinks (Xiao et al., 2025). A token’s importance can therefore differ across heads, motivating retention decisions specific to each layer and KV head.

2.2 SOFT TOKEN COMPRESSION

Soft compression represents a long context through a shorter sequence of continuous representations. AutoCompressors learn summary vectors that condition subsequent language modeling (Chevalier et al., 2023). The In-context Autoencoder (ICAE) maps context to compact memory slots using reconstruction and language-model pretraining followed by instruction fine-tuning (Ge et al., 2024). Latent Context Language Models (LCLMs) train encoder–decoder compressors at scale and use the

resulting latent tokens as context for a decoder (Li et al., 2026). RestoreKV complements retained KV entries with a small learned restore cache, trained against full-cache answer distributions while keeping the evictor fixed (Baek et al., 2026). These approaches synthesize representations that can combine information from multiple tokens. Our approach learns which original KV entries to retain. Message passing changes the selector’s representations while preserving the retained keys and values.

2.3 HARD TOKEN COMPRESSION

Compression can also operate on discrete text or internal model states. At the text level, summarization rewrites a history into a shorter description, whereas extractive prompt compression removes portions of the original input. LLMingua performs budgeted, iterative token deletion before the target model processes the prompt (Jiang et al., 2023). Pruning can instead occur within a model: FastV, for example, removes visual-token representations after early layers of a vision–language model (Chen et al., 2024). Such approaches differ in what is removed and when the compression cost is incurred.

KV eviction directly removes stored key–value pairs. H₂O combines accumulated attention importance with recency (Zhang et al., 2023); StreamingLLM retains initial attention sinks and a recent window for bounded-memory streaming (Xiao et al., 2024); and SnapKV selects important positions using attention from a prompt-end observation window (Li et al., 2024). Maintaining fluent streaming generation and preserving evidence for arbitrary future questions are different objectives. KVzip explicitly addresses query-agnostic cache reuse by scoring KV entries through context reconstruction (Kim et al., 2025). Fast KVzip learns lightweight gates to predict these scores from hidden states (Kim et al., 2026), and KVzap also learns an efficient approximation to KVzip (Jegou & Jeblick, 2026). Learned eviction has additionally been optimized through generation objectives: DMS uses output-distribution supervision (Łańcucki et al., 2025), while TRIM-KV trains retention gates with prediction and distribution-matching losses (Bui et al., 2026). Our method builds on Fast KVzip’s scoring framework, adding relational token modeling and a reconstruction-supervised warm-up followed by supervision from full-cache reference answers.

Graph-based selection. Message-passing neural networks update node representations by aggregating information from their neighbors (Gilmer et al., 2017). Graph Isomorphism Networks (GINs) combine a node’s own representation with summed neighborhood features before a learned transformation (Xu et al., 2019), motivating our aggregation-and-update design. Graph-based KV selection also has direct precedent: GraphKV (Li et al., 2025a) propagates similarity-based decay signals from selected tokens to discourage redundant retention. Our focus is a learned graph selector whose message-passing transformations and retention scores are optimized through reconstruction supervision and subsequent answer-supervised adaptation.

3 METHOD

We consider a frozen autoregressive Transformer with L layers and H KV heads. Prefilling a context of T tokens, $c = (c_1, \dots, c_T)$, produces a cache $\mathcal{C}(c)$ and hidden representations $X^\ell \in \mathbb{R}^{T \times d}$. Compression is applied after this prefill, producing a cache for subsequent questions. A selector with parameters θ assigns a score $s_{\ell hi}$ to each context entry. Compression retains a subset of the original KV pairs according to these scores and a retention budget ρ . The selector observes the context before the downstream question is supplied. System-prefix entries and subsequently appended question and answer tokens lie outside the context budget.

3.1 FAST KVZIP GATING MECHANISM

Fast KVzip predicts KV importance from token representations using a low-dimensional sink-attention gate (Kim et al., 2026). Fix a layer ℓ and KV head h , and write $x = x_i^\ell$. Suppressing these indices on the projection parameters, the gate forms a key and G queries, where G is the number of query heads sharing the KV head:

$$\hat{\mathbf{q}}_j = \text{RMSNorm}_\ell^q(xW_j^q + \mathbf{b}_j^q), \quad \hat{\mathbf{k}} = \text{RMSNorm}_\ell^k(xW^k), \quad (3)$$

with $W_j^q, W^k \in \mathbb{R}^{d \times d_g}$ and gate width d_g . RMSNorm rescales each projected vector by its root-mean-square magnitude and applies learned featurewise scales. These are auxiliary gate features.

Let $\kappa_1, \dots, \kappa_S \in \mathbb{R}^{d_g}$ be learned reference keys, shared across token positions. With the token-key logit $a_j = \langle \hat{\mathbf{q}}_j, \hat{\mathbf{k}} \rangle / \sqrt{d_g} + b_j$ and learned scalar bias b_j , the gate used here is

$$s_{\ell hi} = g_{\ell h}(x_i^\ell) = \frac{1}{G} \sum_{j=1}^G \frac{\exp(a_j)}{\exp(a_j) + \sum_{m=1}^S \exp(\langle \hat{\mathbf{q}}_j, \kappa_m \rangle / \sqrt{d_g})}. \quad (4)$$

Each fraction measures the attention mass assigned to the token’s key in competition with the reference keys; averaging yields one importance score per KV head. The frozen Transformer supplies contextual representations, which this gate scores separately at each position. Our graph module enriches these representations through message passing before applying the same mapping $g_{\ell h}$.

3.2 GRAPH-BASED TOKEN MIXING

3.2.1 GRAPH CONSTRUCTION

We partition the context into contiguous blocks $\mathcal{B}$ and apply the same selector parameters to each block. A single block can also cover the entire context. For one block $b \in \mathcal{B}$ of $T_b = |b|$ tokens, suppress the layer and head indices and write its representations as $X \in \mathbb{R}^{T_b \times d}$. Learned projections $W_a, W_v \in \mathbb{R}^{d \times r}$ produce relation features U and message features R :

$$U = XW_a, \quad R = XW_v, \quad A = \frac{1}{T_b} UU^\top. \quad (5)$$

Here A_{ij} is the weight of the connection between tokens i and j . The relation projection is learned across contexts, while the graph is recomputed from the representations of each input. Every layer-KV-head pair has its own projections. The adjacency is symmetric, can contain signed weights, and includes self-connections. These weights define learned feature interactions rather than probabilities or prescribed semantic relations.

3.2.2 MESSAGE PASSING AND RETENTION SCORES

With $W_o \in \mathbb{R}^{r \times d}$, we aggregate messages and form a residual update:

$$P = ARW_o, \quad X' = X + \alpha \text{LeakyReLU}(\text{BatchNorm}(P)), \quad s_{\ell hi} = g_{\ell h}(x_i'). \quad (6)$$

BatchNorm (Ioffe & Szegedy, 2015) uses the current block’s token statistics during both training and inference. The scalar α , normalization parameters, and message projections are specific to each layer-KV-head pair. This update follows the aggregation-and-update structure of message-passing networks (Gilmer et al., 2017; Xu et al., 2019); the learned weighted graph specifies which token features contribute to each message. We evaluate message passing in factorized form without constructing the adjacency matrix, giving a cost linear in block length for fixed feature dimensions (Appendix A.3.1).

The residual changes only the auxiliary representations used for scoring. The language model’s hidden states and retained KV vectors are not replaced by mixed features. Scores from all blocks are concatenated before eviction, so graph blocks do not impose separate retention quotas. Given a retention ratio ρ , a uniform per-head budget retains the $k = \max(1, \lfloor \rho T \rfloor)$ highest-scoring context entries in each layer and KV head. The scores also support a shared budget across layers and heads, allowing nonuniform retention. Selected entries keep their original order and positional information. Block boundaries restrict the selector’s message passing, while its input representations retain the contextual information supplied by the frozen Transformer.

3.3 TWO-STAGE TRAINING

3.3.1 STAGE 1: RECONSTRUCTION-SUPERVISED WARM-UP

KVzip computes an importance target $t_{\ell hi} \in [0, 1]$ from the maximum attention received by a cached entry during context reconstruction, aggregating over reconstruction positions and the query heads

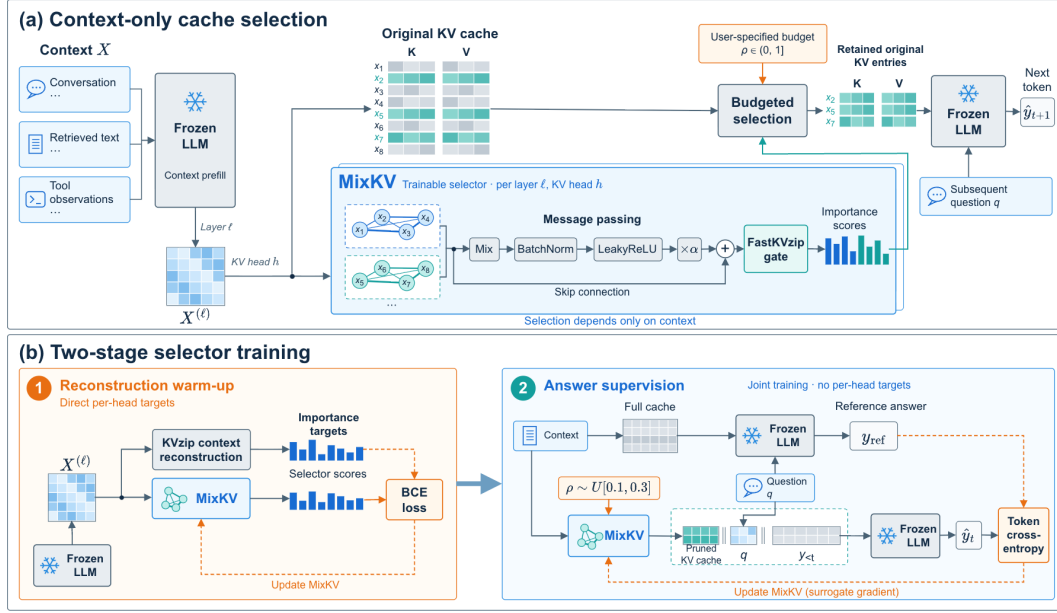


Figure 1: **MixKV architecture and training.** (a) A trainable selector mixes context-token representations through message passing and scores them with a Fast KVzip gate. Budgeted selection retains original KV entries before a subsequent question is supplied. (b) Reconstruction-score supervision warms up the selector, followed by answer supervision from the frozen full-cache model using a straight-through surrogate. The language model remains frozen in both stages.

that share that KV head (Kim et al., 2025). Following Fast KVzip, we use these scores as continuous targets for binary cross-entropy. For a collection of training blocks, the objective is

$$\mathcal{L}_{\text{warm}} = -\frac{1}{|\mathcal{B}|LH} \sum_{b \in \mathcal{B}} \sum_{\ell, h} \frac{1}{T_b} \sum_{i \in b} [t_{\ell hi} \log s_{\ell hi} + (1 - t_{\ell hi}) \log(1 - s_{\ell hi})]. \quad (7)$$

Both the graph mixer and gate are trained, with the gate optionally initialized from Fast KVzip. Reconstruction supervision supplies a direct target for each cache entry and gives the selector an informed starting point. These targets express the reconstruction procedure’s assessment of importance; reproducing them alone does not directly optimize answers generated from a compressed cache.

3.3.2 STAGE 2: ANSWER SUPERVISION

For a context–question pair (c, q) , the frozen language model with its full cache generates a reference answer y^* . We then compress the context cache using the selector and train on the reference answer under teacher forcing:

$$\mathcal{L}_{\text{ans}}(\theta) = -\frac{1}{|y^*|} \sum_{t=1}^{|y^*|} \log p_{\phi}(y_t^* \mid q, y_{<t}^*, \mathcal{C}_{\theta}(c; \rho)), \quad (8)$$

where ϕ denotes the frozen language-model parameters. Only answer tokens contribute prediction loss. The question and reference prefix condition the prediction, and gradients update the mixer and gate. We vary ρ across training examples, either by sampling a retention interval or by following a decreasing schedule. This stage aligns cache selection with the model’s ability to predict its full-cache answers, rather than requiring continued agreement with reconstruction scores. It uses reference sequences, not a loss matching the teacher’s complete output distribution. Answer supervision is query-dependent during training, but the learned selector remains a function of the context alone.

To train through hard top- k selection, we use a straight-through surrogate that preserves the retained KV entries in the forward pass while supplying score gradients during backpropagation (Ap-

pendix A.2). We then recompute the selector one graph block at a time and replay these score gradients, limiting live selector activations to one block (Appendix A.3.2).

4 EXPERIMENTAL RESULTS

4.1 SETUP

Datasets, models, baselines (including a summary baseline where the model is asked to summarize the context and then answers the question based on the summary - watch out for varying retention ratio of the summary)

The reported suite combines RULER (Hsieh et al., 2024) and SCBench (Li et al., 2025b). Appendix C explains every supported task and length variant, summarizes context lengths, and identifies the configurations still missing from the matched three-method comparison. All 27 prepared SCBench configurations are reported, spanning mean context lengths from approximately 7K to 169K Qwen tokens.

4.2 MAIN RESULTS

Tables 1–3 in Appendix B report all 60 evaluated configurations separately for our graph-based selector, Fast KVzip, and KVzip. Each table includes the complete per-task curves, benchmark-level macro-averages, and the method’s own uncompressed baseline; no scores are averaged across retention ratios. Appendix D provides the corresponding retention–score curves, benchmark summaries, and official task-family groupings.

All three methods use Qwen2.5-7B-Instruct-1M and the same prepared benchmark splits: all 13 RULER tasks at both 4K and 8K, seven selected tasks at 64K, and all 27 prepared SCBench configurations (11 base tasks and 16 length variants). Every RULER configuration contains 500 examples. The complete suite contains 18,531 contexts and 28,236 questions per method. The comparison preserves each deployed method’s evaluation procedure: our selector and KVzip prune after full-context prefill, while Fast KVzip uses its native chunked-prefill eviction with 16,000-token chunks. Our selector and Fast KVzip use a protected local window of 2%; KVzip uses no protected window. Thus this is a comparison of method pipelines, not an ablation that isolates the scoring architecture.

Scores are on a 0–100 scale. Aggregate rows are unweighted means of the included configuration scores at each retention ratio. SCBench is summarized both over its 11 base tasks and over all 27 configurations; the latter gives equal weight to each length variant. These and the selected RULER aggregate are descriptive prepared-split summaries, not official complete-benchmark scores. Mixed SCBench tasks retain one configuration score each, and Summary + needles retains the inherited 48-token output cap for both summary and retrieval questions. Full-cache scores are reported separately for each method rather than assumed to be identical.

Context-length comparison. Figure 2 pools individual context examples from all 60 configurations into buckets of their measured token lengths. To account for differences in task difficulty, we divide each method’s mean score within each configuration and bucket by its own full-cache mean on the same contexts. We then average these relative scores with weights proportional to the number of contexts per configuration in the bucket. Thus 100% denotes full-cache performance; values above 100% are retained. Scores use the native evaluator, including the equal-component mean for RepoQA + KV, and retention ratios are plotted separately. This preserves equal context weighting after normalization and differs from the configuration-level macro-averages in the tables. Task composition and sample counts still vary by bucket, so these curves describe the mixed suite rather than isolating the effect of context length.

On the seven RULER tasks shared by our 4K, 8K, and 64K evaluations (single 1, multikey 3, multi-value, VT, CWE, FWE, and QA 2), the score gap to KVzip at 20% retention changes from −3.41 and −3.46 points to +2.81 points, respectively. This suggests stronger relative performance at longer contexts, but not a universal scaling trend: gains over Fast KVzip are mixed, and task composition and retention ratio matter.

4.3 EFFICIENCY

Show time taken from prefill start to the point we have pruned KV cache (just before generation). Compare to other methods like KVzip, FastKVzip, and summarizing the context.

4.4 SUMMARIZATION ALIGNMENT TO THE UNDERLYING LLM

CHECK IF TRUE: Most benchmarks are needle in a haystack tasks but fail to say if the model really captures the essence of the context as a whole.

Evaluate on summarizing long texts (same context, question is "summarize this text") - compare full cache model answer with pruned cache answer - checks if the model stays in the same distribution. Measure with summarization metrics (rouge, bleu, llm as a judge)

4.5 DECODING INTENSIVE TASKS

Take benchmarks when the model needs to reason in many tokens and do the compaction every X tokens. This is modeling a scenario of an agent running in a long session.

4.6 WHAT GRAPH STRUCTURE IS BEING LEARNED

Find a mathematical metric to report, or some figures that can tell something about what relations are being learned in every head and how similar the structures are across different grouped-query heads, heads, and different layers.

4.7 ABLATIONS

4.7.1 EFFECT OF 2-STAGE TRAINING

Train only stage 1, show results.

Train only stage 2, show results.

4.7.2 EFFECT OF GNN

Train only stage 1 our model, show results compared to FastKVzip.

Take the 2 checkpoints (ours and FastKVzip) and train stage 2 on top of the pretrained checkpoint. See what contributes more - the architecture or the training methodology.

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, “we checked LLM-generated research ideas for potential plagiarism through a manual literature survey”, “LLM-generated code was verified and tested for correctness by 2 authors”, etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

AUTHOR CONTRIBUTIONS

If you’d like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4895–4901, 2023. doi: 10.18653/v1/2023.emnlp-main.298. URL <https://aclanthology.org/2023.emnlp-main.298/>.

- Changwoo Baek, Seungjun Shin, and Kyeongbo Kong. RestoreKV: Recovering full-cache behavior under aggressive query-agnostic KV cache eviction. *arXiv preprint arXiv:2608.01247*, 2026. URL <https://arxiv.org/abs/2608.01247>.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. URL <https://arxiv.org/abs/1308.3432>.
- Ngoc Bui, Shubham Sharma, Simran Lamba, Saumitra Mishra, and Rex Ying. Cache what lasts: Token retention for memory-bounded KV cache in LLMs. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2512.03324>.
- Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. *arXiv preprint arXiv:2403.06764*, 2024. URL <https://arxiv.org/abs/2403.06764>.
- Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016. URL <https://arxiv.org/abs/1604.06174>.
- Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3829–3846, 2023. doi: 10.18653/v1/2023.emnlp-main.232. URL <https://aclanthology.org/2023.emnlp-main.232/>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2307.06945>.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1263–1272, 2017. URL <https://proceedings.mlr.press/v70/gilmer17a.html>.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krirman, Shantanu Acharya, Dima Reakesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling*, 2024. URL <https://arxiv.org/abs/2404.06654>.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 448–456. PMLR, 2015. URL <https://proceedings.mlr.press/v37/ioffe15.html>.
- Simon Jegou and Maximilian Jeblick. KVzap: Fast, adaptive, and faithful KV cache pruning. *arXiv preprint arXiv:2601.07891*, 2026. URL <https://arxiv.org/abs/2601.07891>.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13358–13376, 2023. doi: 10.18653/v1/2023.emnlp-main.825. URL <https://aclanthology.org/2023.emnlp-main.825/>.
- Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL <https://arxiv.org/abs/2505.23416>.

- Jang-Hyun Kim, Dongyoon Han, and Sangdoo Yun. Fast KVzip: Efficient and accurate LLM inference with gated KV eviction. *arXiv preprint arXiv:2601.17668*, 2026. URL <https://arxiv.org/abs/2601.17668>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. doi: 10.1145/3600006.3613165. URL <https://arxiv.org/abs/2309.06180>.
- Adrian Łańcucki, Konrad Staniszewski, Piotr Nawrot, and Edoardo M. Ponti. Inference-time hyper-scaling with KV cache compression. *arXiv preprint arXiv:2506.05345*, 2025. URL <https://arxiv.org/abs/2506.05345>.
- Ang Li, Sean McLeish, Haozhe Chen, Nimit Kalra, Zaiqian Chen, Artem Gazizov, Venkata Anoop Suhas Kumar Morisetty, Bhavya Kailkhura, Harshitha Menon, Zhuang Liu, Brian R. Bartoldson, Tom Goldstein, Sanae Lotfi, Micah Goldblum, and Pavel Izmailov. End-to-end context compression at scale. *arXiv preprint arXiv:2606.09659*, 2026. URL <https://arxiv.org/abs/2606.09659>.
- Xuelin Li, Xiangqi Jin, and Linfeng Zhang. GraphKV: Breaking the static selection paradigm with graph-based KV cache eviction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 21899–21909, 2025a. doi: 10.18653/v1/2025.emnlp-main.1112. URL <https://aclanthology.org/2025.emnlp-main.1112/>.
- Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. SCBench: A KV cache-centric analysis of long-context methods. In *International Conference on Learning Representations*, 2025b. URL <https://arxiv.org/abs/2412.10319>.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/28ab418242603e0f7323e54185d19bde-Abstract-Conference.html.
- NVIDIA. RULER: Official synthetic task configurations. <https://github.com/NVIDIA/RULER/blob/main/scripts/synthetic.yaml>, 2024. Accessed 9 September 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2383–2392, 2016. doi: 10.18653/v1/D16-1264. URL <https://aclanthology.org/D16-1264/>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 5797–5808, 2019. doi: 10.18653/v1/P19-1580. URL <https://aclanthology.org/P19-1580/>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.

- Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.10819>.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1810.00826>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.14048>.

A ADDITIONAL METHOD DETAILS

A.1 BATCHNORM AND GRAPH SCOPE

For a block b with $T_b = |b|$ tokens, the BatchNorm operator in Equation 6 computes a population mean and variance separately for each output feature j :

$$\mu_j = \frac{1}{T_b} \sum_{i=1}^{T_b} P_{ij}, \quad \sigma_j^2 = \frac{1}{T_b} \sum_{i=1}^{T_b} (P_{ij} - \mu_j)^2, \quad \text{BatchNorm}(P)_{ij} = \gamma_j \frac{P_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j. \quad (9)$$

We use $\epsilon = 10^{-5}$ for numerical stability. Means and variances are recomputed on the current block at inference; there are no running statistics. The featurewise affine parameters γ, β and scalar residual coefficient α are learned for each layer–KV-head pair. Normalization and its affine transform precede LeakyReLU. The original representation is added outside the activation.

The graph adjacency $UU^\top/T_b$ is positive semidefinite, but its individual edge weights can be negative. Its diagonal is retained. Our update is a weighted message-passing operator with a learned residual; it is not the standard GIN architecture, and GIN’s expressivity results do not establish an equivalent guarantee for this selector (Xu et al., 2019). Independent graph blocks restrict which positions exchange new messages. This architectural choice is distinct from dividing the computation of a fixed graph into smaller token batches, which preserves that graph’s normalization domain.

A.2 LEARNING THROUGH DISCRETE CACHE SELECTION

Hard top- k selection does not provide a useful ordinary derivative with respect to the scores. We use a straight-through surrogate (Bengio et al., 2013) that leaves the compacted cache unchanged in the forward pass. Suppressing layer and head indices, define scaled softmax weights

$$a_i = k \frac{\exp(s_i/\tau)}{\sum_{j=1}^T \exp(s_j/\tau)}, \quad \tilde{K}_i = K_i, \quad \tilde{V}_i = [1 + a_i - \text{sg}(a_i)] V_i, \quad \text{for retained } i, \quad (10)$$

where sg is the identity in the forward pass with zero derivative, and $\tau > 0$ controls the surrogate’s smoothness. The multiplier is exactly one during prediction, and only selected entries are present. During backpropagation, the value multipliers provide a gradient to the scores. The shared softmax denominator also gives unselected scores an indirect competitive signal. The selected keys and discrete indices have no score-gradient path. This is a biased surrogate for learning a discrete selection policy; it is not an exact derivative of top- k .

Score gradient of the surrogate. Let $\mathcal{I}$ denote the retained indices for one layer and KV head, and let $\pi_i = \text{softmax}(s/\tau)_i$, so that $a_i = k\pi_i$ in Equation 10. The weights a_i sum to k and can exceed one; they are not Bernoulli retention probabilities. For a selected entry, define

$$c_i = \left\langle \frac{\partial \mathcal{L}_{\text{ans}}}{\partial \tilde{V}_i}, V_i \right\rangle, \quad i \in \mathcal{I}. \quad (11)$$

Set $c_j = 0$ for unselected entries. Since $\partial a_i / \partial s_j = (a_i/\tau)(\mathbf{1}_{i=j} - \pi_j)$, the surrogate score gradient is

$$\frac{\partial \mathcal{L}_{\text{ans}}}{\partial s_j} = \frac{1}{\tau} \left[\mathbf{1}_{j \in \mathcal{I}} c_j a_j - \pi_j \sum_{i \in \mathcal{I}} c_i a_i \right]. \quad (12)$$

The first term measures sensitivity through a retained value. The second couples all scores through softmax normalization, including scores of evicted entries. An evicted score can therefore receive feedback even though its value is absent from the forward cache. This feedback is indirect: it does not evaluate what the language model would have generated had that evicted entry been retained. The temperature changes this backward surrogate, not the temperature used to generate reference answers.

A.3 EFFICIENCY AND GRADIENT REPLAY

A.3.1 FACTORIZED MESSAGE PASSING

The graph in Equation 5 need not be materialized. Associativity gives

$$P = U \left(\frac{U^\top R}{T_b} W_o \right). \quad (13)$$

The intermediate contraction is $r \times r$, avoiding storage of a $T_b \times T_b$ adjacency. For one layer-head pair, projection and message computation require $O(T_b dr + T_b r^2 + r^2 d)$ operations, linear in block length for fixed feature widths. Tokenwise scoring adds another linear pass. This bound concerns the selector’s message computation; it does not include cache selection or imply linear-time Transformer prefill.

Forward workspace. For forward scoring without retained activation graphs, one active layer-head pair can retain U with $O(T_b r)$ entries, the $r \times r$ contraction, and the $r \times d$ output kernel. Normalization statistics require $O(d)$ storage. Streaming the remaining feature-width operations in batches of m tokens requires $O(md)$ additional workspace, rather than storing every intermediate of size $T_b \times d$. With blocks of maximum size B , the message computation across a context costs

$$O(LH [Tdr + Tr^2 + \lceil T/B \rceil r^2 d]). \quad (14)$$

This expression excludes the gate, cache selection, and frozen-model computation. The graph mixer has $3dr + 2d + 1$ learned parameters per layer-KV-head pair, independently of context length.

A.3.2 REPLAYING SELECTOR GRADIENTS

Although the language model is frozen, answer supervision still requires differentiating its predictions with respect to the compressed cache. Keeping the selector’s activation graph alive during that backward pass adds avoidable memory pressure. Write the concatenated score function as $s = \text{concat}_{b \in \mathcal{B}} f_\theta(X_b)$, where f_θ is the block scorer. We first compute all selector scores without retaining their activation graph, then differentiate the answer loss with respect to a detached score tensor to obtain $u = \partial \mathcal{L}_{\text{ans}} / \partial s$. After the language-model backward pass, we recompute the selector one independent block at a time and apply the saved score gradients:

$$\nabla_\theta \mathcal{L}_{\text{ans}} = \sum_{b \in \mathcal{B}} J_{f_\theta(X_b)}^\top u_b. \quad (15)$$

Here J is the block scorer’s Jacobian with respect to θ ; it is applied without being explicitly formed. The answer loss can couple scores from all blocks through selection and the surrogate, but this cross-block competition is already represented in u . Once the full score derivative is available, the chain rule separates the remaining parameter gradient by independent scorer blocks. Each block is recomputed with the same parameters and normalization statistics as in the first pass, and only its corresponding slice u_b is applied. Gradients accumulate across all blocks before the parameters are updated. With unchanged parameters and deterministic replay, this is an exact chain-rule rearrangement of the chosen surrogate gradient. Arbitrarily slicing a graph that contains cross-block edges would not satisfy this independence argument.

Replay trades an additional selector forward pass for retaining at most one block’s scorer activation graph, following the recomputation principle of activation checkpointing (Chen et al., 2016). It does not repeat context prefill or the language-model backward pass for each block.

Full-context memory. Replay still requires $O(LHT)$ scalar scores and score gradients, the frozen hidden representations, and the original and compacted caches, all of which scale with context length, in addition to language-model answer activations and optimizer state. Consequently, this technique does not make total training memory independent of context length. A full-context graph can use the same replay separation, but its single block spans all T tokens.

Streaming reconstruction-supervised gradients. The warm-up objective also permits recomputation within a fixed graph, provided normalization derivatives account for all its tokens. With

$Z = (P - \mu)/\sqrt{\sigma^2 + \epsilon}$ and $D = \partial\mathcal{L}_{\text{warm}}/\partial Z$, the featurewise backward map is

$$\frac{\partial\mathcal{L}_{\text{warm}}}{\partial P} = \frac{1}{\sqrt{\sigma^2 + \epsilon}} [D - \text{mean}_i(D) - Z \odot \text{mean}_i(D \odot Z)]. \quad (16)$$

The reductions are over the complete graph, even when projection and gate computations are replayed in smaller batches. This preserves the objective’s normalization domain. Storage used by these gradient buffers is additional to the forward workspace above; a bound on factorized forward intermediates is not a bound on all training memory.

B DETAILED BENCHMARK RESULTS

Each table contains all 60 evaluated configurations. Bold aggregate rows are arithmetic means of unrounded configuration scores, computed independently for each column. SCBench 11-base-task mean excludes the 16 length variants; SCBench 27-configuration mean includes them equally. RULER selected 33 weights the 4K, 8K, and selected 64K groups by 13/33, 13/33, and 7/33. These are descriptive prepared-split summaries, not official complete-benchmark scores; the overall mean also mixes heterogeneous metrics. Display values are rounded only after aggregation. Full-cache scores are method-specific.

Table 1: Our graph-based selector: post-prefill pruning; 2% protected local window. Absolute scores (0–100) for the complete selected suite.

Benchmark / configuration	20%	30%	40%	50%	75%	Full (100%)
SCBench KV	71.60	69.60	72.00	72.40	68.60	68.20
SCBench KV (mid)	63.20	68.60	70.40	70.60	69.20	68.90
SCBench KV (short)	70.50	83.40	85.30	86.30	84.70	85.50
SCBench KV (tiny)	68.20	85.90	89.70	90.20	88.90	88.00
SCBench MF	35.33	37.33	35.83	33.50	33.67	33.00
SCBench MF (mid)	35.50	36.83	35.00	32.67	34.33	34.83
SCBench MF (short)	38.67	37.83	36.50	34.67	37.00	37.83
SCBench MF (tiny)	40.00	40.50	39.00	37.33	38.33	39.50
SCBench QA (English)	42.51	45.64	42.85	45.00	44.02	42.92
SCBench RepoQA	55.68	58.64	59.55	59.55	59.77	59.09
SCBench RepoQA (short)	70.00	70.00	70.91	70.91	72.73	72.73
SCBench RepoQA (tiny)	64.44	73.33	75.56	71.11	75.56	75.56
SCBench Summary	36.05	36.30	36.23	36.51	36.77	36.82
SCBench Summary (mid)	40.36	40.60	39.76	39.94	39.95	39.17
SCBench Summary (short)	40.33	40.52	39.06	40.51	40.63	39.31
SCBench Summary (tiny)	38.08	38.21	36.95	38.29	38.34	37.12
SCBench Choice (English)	72.22	73.61	80.56	80.56	76.39	79.17
SCBench Many-shot	34.07	34.44	37.04	38.15	38.15	38.52
SCBench Many-shot (short)	32.96	33.70	35.56	36.67	34.81	35.56
SCBench Many-shot (tiny)	33.70	34.81	36.30	34.81	35.56	35.19
SCBench Prefix/suffix	30.00	42.80	43.40	40.00	44.80	51.20
SCBench Prefix/suffix (mid)	42.20	58.00	65.60	65.80	70.20	69.60
SCBench Prefix/suffix (short)	34.40	61.40	76.80	81.60	86.60	88.80
SCBench Prefix/suffix (tiny)	62.40	82.40	87.80	88.80	90.60	91.40
SCBench VT	42.31	40.44	41.07	41.78	41.38	41.29
SCBench Summary + needles	65.47	67.63	67.97	67.92	68.18	68.35
SCBench RepoQA + KV	74.86	79.26	80.11	80.40	81.25	80.82
SCBench 11-base-task mean	50.92	53.25	54.24	54.16	53.91	54.49
SCBench 27-configuration mean	49.45	54.51	56.18	56.15	56.68	56.98

Continued on next page

Table 1 (continued)

Benchmark / configuration	20%	30%	40%	50%	75%	Full (100%)
RULER 4K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 4K NIAH single 2	100.00	100.00	100.00	100.00	100.00	100.00
RULER 4K NIAH single 3	99.40	99.60	99.80	99.80	99.60	98.80
RULER 4K NIAH multikey 1	96.00	98.40	98.60	99.20	99.20	99.00
RULER 4K NIAH multikey 2	98.20	99.40	99.40	99.60	99.80	99.80
RULER 4K NIAH multikey 3	98.00	98.80	98.80	99.40	99.60	99.60
RULER 4K NIAH multivalue	66.30	73.70	82.70	90.75	93.90	92.20
RULER 4K NIAH multiquery	99.50	99.70	99.95	99.90	99.90	100.00
RULER 4K VT	94.24	95.12	96.68	96.96	98.64	98.88
RULER 4K CWE	93.32	97.08	97.46	97.90	98.34	98.62
RULER 4K FWE	80.33	83.13	81.73	81.67	84.67	85.80
RULER 4K QA 1	80.40	83.40	86.20	85.60	85.20	85.20
RULER 4K QA 2	60.40	60.80	61.80	59.60	58.80	59.60
RULER 4K 13-task mean	89.70	91.47	92.55	93.11	93.67	93.65
RULER 8K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 8K NIAH single 2	100.00	100.00	100.00	100.00	100.00	100.00
RULER 8K NIAH single 3	99.60	99.40	99.60	99.60	99.60	99.40
RULER 8K NIAH multikey 1	96.00	98.20	98.80	98.80	99.40	99.60
RULER 8K NIAH multikey 2	97.20	98.60	98.80	99.00	99.80	99.80
RULER 8K NIAH multikey 3	90.80	95.40	95.80	95.80	97.00	97.20
RULER 8K NIAH multivalue	61.90	74.65	78.75	86.50	85.90	84.70
RULER 8K NIAH multiquery	99.60	99.75	99.85	99.95	99.95	99.90
RULER 8K VT	89.24	88.96	94.24	94.92	97.36	97.88
RULER 8K CWE	86.80	91.26	90.58	90.70	90.70	91.02
RULER 8K FWE	73.27	81.73	82.47	81.73	82.67	83.20
RULER 8K QA 1	75.40	79.20	79.80	80.40	81.40	81.40
RULER 8K QA 2	56.40	58.20	57.60	56.00	55.20	55.00
RULER 8K 13-task mean	86.63	89.64	90.48	91.03	91.46	91.47
RULER 64K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 64K NIAH multikey 3	89.00	90.40	92.40	91.80	91.80	91.60
RULER 64K NIAH multivalue	64.05	76.85	80.60	86.75	85.35	82.20
RULER 64K VT	80.28	78.80	79.32	80.28	78.32	77.04
RULER 64K CWE	36.74	42.80	42.08	38.50	27.74	33.80
RULER 64K FWE	80.93	85.53	86.47	87.07	86.53	86.00
RULER 64K QA 2	47.60	49.20	48.00	48.60	47.60	46.60
RULER 64K selected 7 mean	71.23	74.80	75.55	76.14	73.91	73.89
RULER selected 33 mean	84.57	87.21	88.13	88.69	88.61	88.60
Overall selected 60 mean	68.77	72.50	73.75	74.05	74.24	74.37

Table 2: Fast KVzip: native chunked-prefill eviction; 2% protected local window.
Absolute scores (0–100) for the complete selected suite.

Benchmark / configuration	20%	30%	40%	50%	75%	Full (100%)
SCBench KV	47.00	66.80	66.40	72.80	68.20	68.20
SCBench KV (mid)	61.80	70.10	72.50	71.00	68.90	68.90
SCBench KV (short)	71.10	83.10	85.40	85.70	85.70	85.20
SCBench KV (tiny)	65.90	83.20	90.10	90.50	89.40	88.10
SCBench MF	32.83	33.50	34.00	33.83	32.83	33.00
SCBench MF (mid)	34.33	33.00	34.50	35.17	34.17	34.83
SCBench MF (short)	38.17	35.33	34.50	33.83	35.83	37.83
SCBench MF (tiny)	41.33	39.33	37.00	38.00	37.83	39.83
SCBench QA (English)	44.20	44.74	42.02	41.76	42.54	41.52
SCBench RepoQA	57.73	59.32	60.23	61.36	58.18	59.09
SCBench RepoQA (short)	69.09	67.27	70.00	71.82	72.73	72.73
SCBench RepoQA (tiny)	71.11	73.33	75.56	75.56	75.56	75.56
SCBench Summary	35.97	36.82	36.49	36.80	36.83	36.82
SCBench Summary (mid)	40.60	39.88	40.03	39.55	38.76	39.17
SCBench Summary (short)	40.60	39.88	40.03	39.55	38.76	39.17
SCBench Summary (tiny)	38.45	38.19	37.04	37.21	37.12	37.12
SCBench Choice (English)	73.61	75.00	75.00	75.00	79.17	79.17
SCBench Many-shot	33.70	36.67	37.04	36.30	36.30	38.52
SCBench Many-shot (short)	34.44	35.93	34.81	34.07	34.07	35.56
SCBench Many-shot (tiny)	34.07	34.44	32.96	33.70	35.19	35.19
SCBench Prefix/suffix	38.80	48.40	55.60	47.40	36.80	51.20
SCBench Prefix/suffix (mid)	47.80	63.20	74.80	70.80	67.20	69.60
SCBench Prefix/suffix (short)	45.00	62.00	80.20	85.00	88.00	88.80
SCBench Prefix/suffix (tiny)	64.00	80.40	88.80	90.00	92.00	91.40
SCBench VT	43.51	41.73	40.58	40.53	41.38	41.29
SCBench Summary + needles	65.64	68.29	67.86	67.56	68.17	68.35
SCBench RepoQA + KV	77.84	79.55	80.40	80.26	80.68	80.82
SCBench 11-base-task mean	50.08	53.71	54.15	53.96	52.83	54.36
SCBench 27-configuration mean	49.95	54.42	56.44	56.48	56.01	56.92

Continued on next page

Table 2 (continued)

Benchmark / configuration	20%	30%	40%	50%	75%	Full (100%)
RULER 4K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 4K NIAH single 2	100.00	100.00	100.00	100.00	100.00	100.00
RULER 4K NIAH single 3	99.40	99.60	99.40	99.80	99.40	99.00
RULER 4K NIAH multikey 1	96.80	98.40	98.80	99.00	99.00	98.80
RULER 4K NIAH multikey 2	98.80	99.40	99.80	99.80	100.00	99.80
RULER 4K NIAH multikey 3	97.60	98.60	98.60	99.60	99.60	99.60
RULER 4K NIAH multivalue	64.25	81.20	83.30	88.45	92.90	91.90
RULER 4K NIAH multiquery	99.50	99.95	99.75	99.80	99.95	100.00
RULER 4K VT	97.40	97.44	98.12	97.60	98.52	98.88
RULER 4K CWE	94.20	97.10	97.64	97.68	98.42	98.62
RULER 4K FWE	79.67	82.00	81.27	81.80	84.73	85.60
RULER 4K QA 1	79.40	83.60	84.80	84.20	85.00	85.20
RULER 4K QA 2	58.00	60.60	60.60	59.60	59.20	59.60
RULER 4K 13-task mean	89.62	92.15	92.47	92.87	93.59	93.62
RULER 8K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 8K NIAH single 2	100.00	100.00	100.00	100.00	100.00	100.00
RULER 8K NIAH single 3	99.80	99.60	99.40	99.60	99.40	99.60
RULER 8K NIAH multikey 1	95.80	98.40	99.00	99.40	99.60	99.60
RULER 8K NIAH multikey 2	98.60	99.20	99.40	99.60	99.80	100.00
RULER 8K NIAH multikey 3	88.00	95.20	96.00	96.60	96.80	97.60
RULER 8K NIAH multivalue	61.20	77.90	80.30	82.70	83.45	84.40
RULER 8K NIAH multiquery	99.55	99.95	99.90	99.95	99.95	99.95
RULER 8K VT	93.68	95.52	96.68	95.92	97.68	98.04
RULER 8K CWE	88.32	91.98	92.00	91.04	90.66	90.88
RULER 8K FWE	79.07	81.53	81.20	82.20	82.47	82.73
RULER 8K QA 1	77.00	78.60	80.80	81.20	82.20	81.60
RULER 8K QA 2	55.80	55.20	57.20	56.00	54.80	54.40
RULER 8K 13-task mean	87.45	90.24	90.91	91.09	91.29	91.45
RULER 64K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 64K NIAH multikey 3	90.80	90.80	92.00	90.80	91.00	91.60
RULER 64K NIAH multivalue	63.95	80.55	86.90	87.60	84.90	82.20
RULER 64K VT	78.96	80.08	84.36	83.88	77.52	77.04
RULER 64K CWE	32.38	43.60	33.72	28.04	28.00	33.58
RULER 64K FWE	80.93	85.20	85.13	85.93	86.60	86.00
RULER 64K QA 2	50.20	50.20	48.80	47.40	47.60	45.60
RULER 64K selected 7 mean	71.03	75.78	75.84	74.81	73.66	73.72
RULER selected 33 mean	84.82	87.92	88.33	88.34	88.46	88.54
Overall selected 60 mean	69.13	72.85	73.98	74.00	73.86	74.31

Table 3: KVzip: native post-prefill reconstruction scoring; no protected local window. Absolute scores (0–100) for the complete selected suite.

Benchmark / configuration	20%	30%	40%	50%	75%	Full (100%)
SCBench KV	36.80	68.40	67.80	66.60	66.40	68.20
SCBench KV (mid)	42.50	65.80	70.80	69.90	67.90	68.90
SCBench KV (short)	63.90	80.40	85.90	85.50	84.30	85.50
SCBench KV (tiny)	72.60	84.60	88.20	89.00	88.90	88.00
SCBench MF	35.33	35.17	34.17	34.17	32.67	33.00
SCBench MF (mid)	34.67	34.33	34.00	34.50	35.00	34.83
SCBench MF (short)	38.83	36.00	35.67	35.67	37.50	37.83
SCBench MF (tiny)	42.17	39.50	38.00	38.17	38.00	39.83
SCBench QA (English)	42.58	46.37	43.96	43.34	40.31	41.52
SCBench RepoQA	51.82	57.27	59.09	58.18	59.09	59.09
SCBench RepoQA (short)	67.27	67.27	70.00	70.91	72.73	72.73
SCBench RepoQA (tiny)	68.89	71.11	71.11	75.56	75.56	75.56
SCBench Summary	35.58	36.00	36.37	36.46	36.94	36.82
SCBench Summary (mid)	39.78	39.30	39.88	39.02	39.49	39.31
SCBench Summary (short)	39.78	39.30	39.88	39.02	39.49	39.31
SCBench Summary (tiny)	37.58	37.11	37.67	36.91	37.28	37.12
SCBench Choice (English)	72.22	77.78	76.39	77.78	77.78	79.17
SCBench Many-shot	33.33	34.44	35.93	36.67	37.41	38.52
SCBench Many-shot (short)	33.33	32.22	33.70	34.07	34.07	35.56
SCBench Many-shot (tiny)	35.93	34.07	33.33	34.44	35.56	35.19
SCBench Prefix/suffix	2.80	23.00	33.00	41.00	50.20	51.20
SCBench Prefix/suffix (mid)	8.60	43.40	59.00	65.80	69.40	69.60
SCBench Prefix/suffix (short)	19.20	54.20	78.40	85.00	87.60	88.80
SCBench Prefix/suffix (tiny)	49.80	78.20	87.60	90.80	91.00	91.40
SCBench VT	40.67	40.84	40.71	40.80	40.31	41.29
SCBench Summary + needles	66.09	67.50	68.13	68.10	68.41	68.35
SCBench RepoQA + KV	73.15	79.26	79.83	79.83	81.11	80.82
SCBench 11-base-task mean	44.58	51.46	52.31	52.99	53.69	54.36
SCBench 27-configuration mean	43.90	51.96	54.76	55.82	56.46	56.94

Continued on next page

Table 3 (continued)

Benchmark / configuration	20%	30%	40%	50%	75%	Full (100%)
RULER 4K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 4K NIAH single 2	100.00	100.00	100.00	100.00	100.00	100.00
RULER 4K NIAH single 3	99.20	99.60	99.80	99.80	99.20	98.80
RULER 4K NIAH multikey 1	97.40	99.00	99.20	99.20	99.00	99.00
RULER 4K NIAH multikey 2	98.80	99.40	99.80	99.80	99.80	99.80
RULER 4K NIAH multikey 3	97.20	99.00	99.20	99.40	99.60	99.60
RULER 4K NIAH multivalue	83.00	87.40	91.20	92.35	94.30	91.90
RULER 4K NIAH multiquery	99.40	99.70	99.80	99.80	100.00	100.00
RULER 4K VT	97.28	97.80	98.44	98.52	98.80	98.64
RULER 4K CWE	94.66	97.18	97.78	97.86	98.44	98.70
RULER 4K FWE	83.33	82.53	82.53	83.60	85.13	85.80
RULER 4K QA 1	83.00	85.00	84.60	84.60	84.80	85.20
RULER 4K QA 2	61.00	60.80	59.40	60.40	59.20	59.60
RULER 4K 13-task mean	91.87	92.88	93.21	93.49	93.71	93.62
RULER 8K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 8K NIAH single 2	100.00	100.00	100.00	100.00	100.00	100.00
RULER 8K NIAH single 3	99.60	99.40	99.40	99.60	99.40	99.60
RULER 8K NIAH multikey 1	96.40	98.40	99.40	99.40	99.60	99.60
RULER 8K NIAH multikey 2	97.40	98.60	99.20	99.60	99.80	100.00
RULER 8K NIAH multikey 3	88.40	94.80	95.60	96.20	96.80	97.20
RULER 8K NIAH multivalue	77.45	79.50	81.50	82.45	83.95	84.40
RULER 8K NIAH multiquery	99.80	99.95	99.90	99.90	99.95	99.90
RULER 8K VT	95.88	96.04	96.20	96.88	97.76	97.96
RULER 8K CWE	87.18	91.00	91.02	90.56	90.76	90.88
RULER 8K FWE	75.73	79.87	80.67	81.47	82.40	82.67
RULER 8K QA 1	76.00	78.00	80.60	81.00	81.60	81.60
RULER 8K QA 2	58.00	55.40	55.80	56.40	54.20	54.40
RULER 8K 13-task mean	88.60	90.07	90.71	91.04	91.25	91.40
RULER 64K NIAH single 1	100.00	100.00	100.00	100.00	100.00	100.00
RULER 64K NIAH multikey 3	68.80	89.80	93.20	92.20	91.80	91.60
RULER 64K NIAH multivalue	73.65	88.95	84.90	85.00	82.20	82.20
RULER 64K VT	76.20	73.76	72.68	73.32	74.56	77.04
RULER 64K CWE	38.58	48.22	41.92	32.38	31.16	33.58
RULER 64K FWE	72.73	76.00	84.07	85.87	86.20	86.00
RULER 64K QA 2	49.00	48.00	48.40	48.00	46.20	45.60
RULER 64K selected 7 mean	68.42	74.96	75.02	73.82	73.16	73.72
RULER selected 33 mean	85.61	87.97	88.37	88.35	88.38	88.52
Overall selected 60 mean	66.84	71.77	73.25	73.71	74.02	74.31

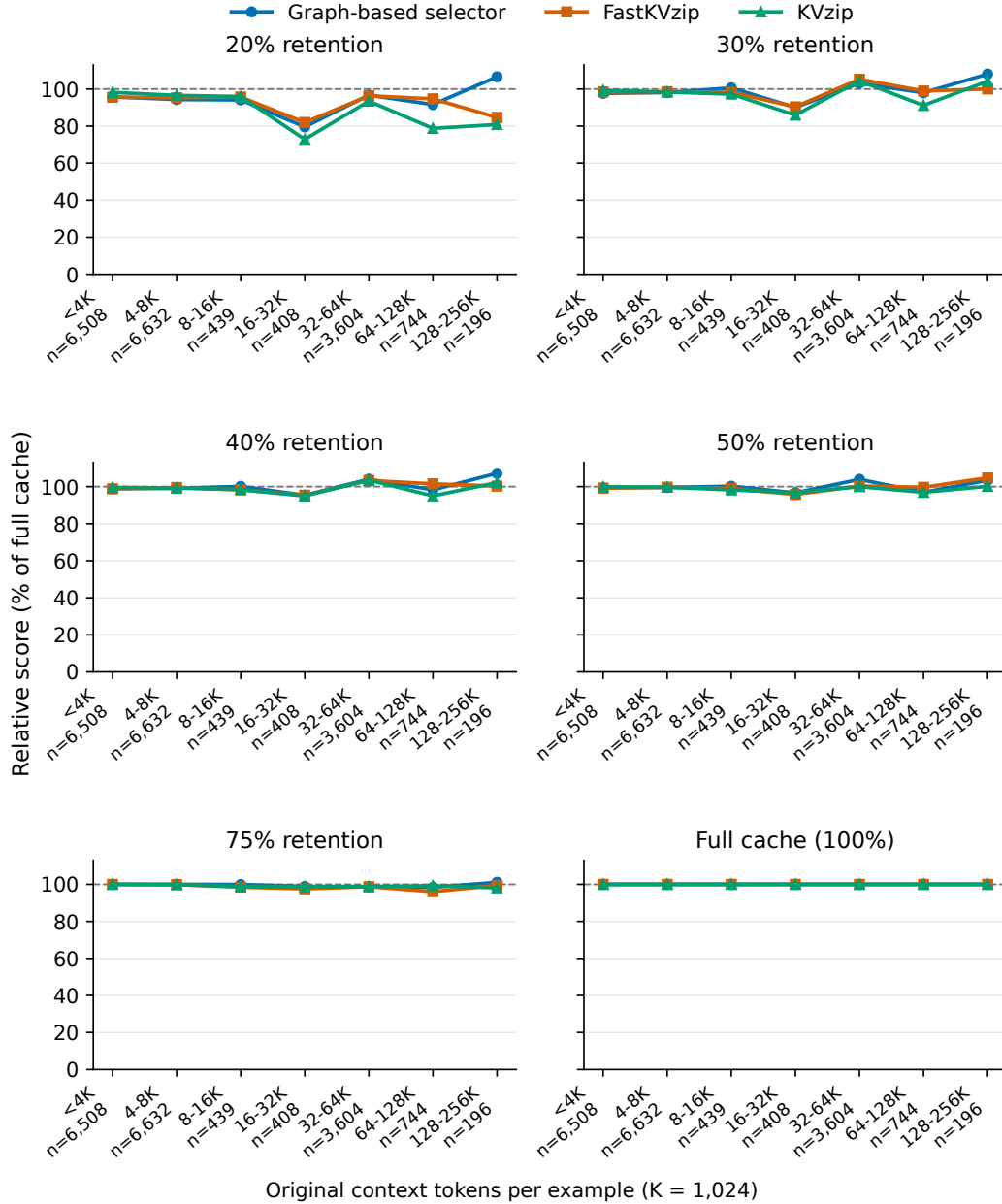


Figure 2: Full-cache-relative score by original context length, pooling all 18,531 matched context examples from the 60 evaluated RULER and SCBench configurations. The same examples enter all three methods at every retention ratio. Within each configuration and bucket, each method’s mean native-evaluator score is divided by its own full-cache mean on the same contexts. These ratios are averaged with context-count weights and multiplied by 100; no retention-ratio averaging is applied. All full-cache denominators are positive. Dashed lines mark 100% (full-cache performance); values above 100% indicate improvement over that baseline. Lengths are measured before compression and exclude system, question, and chat-suffix tokens. Buckets include their lower bound and exclude their upper bound; K denotes 1,024 tokens, and n is the number of contexts in a bucket. Our selector and FastKVzip use a 2% protected window; KVzip uses none. The full-cache panel is 100% by construction. Task composition still varies across buckets, so the curves are descriptive rather than a controlled context-length ablation.

C BENCHMARK GUIDE AND EVALUATION COVERAGE

The current report covers 60 of 107 configured variants. The remaining 47 need matched three-method results: 45 RULER, SQuAD, and filtered GSM8K. This does not imply that they were never run in earlier single-method experiments.

RULER. The 13 tasks in Table 4 cover retrieval, tracing, aggregation, and question answering (Hsieh et al., 2024). We use the standard 4K, 8K, 16K, 32K, 64K, and 128K context-length labels. All tasks are reported at 4K and 8K; starred tasks are also reported at 64K. The other six 64K tasks and all 16K, 32K, and 128K tasks remain outside the matched comparison.

Table 4: RULER task descriptions (Hsieh et al., 2024; NVIDIA, 2024). Prefix identifiers with `ruler_` and append the length label; * marks the seven tasks also reported at 64K.

Task	What it tests
<code>niah_single.1*</code>	Retrieve one numeric passkey associated with a word key, hidden among repetitive filler sentences.
<code>niah_single.2</code>	Retrieve one numeric passkey from natural-language essays instead of repetitive filler.
<code>niah_single.3</code>	Retrieve a UUID-valued needle from essays, testing exact recovery of a longer identifier.
<code>niah_multikey.1</code>	Choose the requested word–number association among four needles embedded in essays; three keys are distractors.
<code>niah_multikey.2</code>	Find a requested word–number association when the context is filled with competing key–value records.
<code>niah_multikey.3*</code>	Retrieve the UUID value for a UUID key from a dense collection of competing records.
<code>niah_multivalue*</code>	Return all four numeric values associated with one key, distributed through an essay context.
<code>niah_multiquery</code>	Answer four key lookups together, requiring recovery of every requested association from the context.
<code>vt*</code>	Follow a four-hop assignment chain and return the five variables that share the requested value.
<code>cwe*</code>	Recover ten unusually common words from a shuffled list; each target occurs more often than distractors.
<code>fwe*</code>	Identify the three most frequent words in a synthetic, skewed-frequency text, requiring global aggregation.
<code>qa.1</code>	Answer a SQuAD-derived question using relevant passages surrounded by unrelated documents.
<code>qa.2*</code>	Answer a HotpotQA-derived question with supporting passages and distractors, testing multi-hop evidence use.

Also remaining are *SQuAD*, Wikipedia paragraph question answering with roughly 160 context tokens, and *GSM8K*, arithmetic word problems with roughly 89 context tokens (Rajpurkar et al., 2016; Cobbe et al., 2021). Our SQuAD adapter uses the training split (18,891 contexts); GSM retains 148 test examples with at least 72 context tokens. Neither is the full official held-out evaluation.

SCBench. SCBench asks multiple questions about a shared context (Li et al., 2025b). Table 5 combines task descriptions with average context lengths for every prepared variant, using SCBench-preprocessed as in KVzip (Kim et al., 2025). All 11 base tasks and 16 prepared length variants now have matched results for all three methods.

Table 5: SCBench tasks and average raw-context Qwen tokens, rounded to the nearest thousand (K). Add `scbench_` to each task and the listed suffix for a variant; *base* has no suffix. All 27 variants are in the current report. System, question, and chat-suffix tokens are excluded.

Task	Avg. tokens	What it tests
kv	base 169K mid 85K short 25K tiny 11K	Retrieve the value for a requested key from a large dictionary of random key-value records.
mf	base 145K mid 73K short 29K tiny 12K	Find requested numerical statistics, such as extrema or the median, in a long array.
qa_eng	base 92K	Answer free-form English questions about a long narrative, locating the relevant evidence in the text.
repoqa	base 67K short 15K tiny 7K	Return the repository function matching a natural-language description, testing semantic code retrieval.
summary	base 105K mid 13K short 13K tiny 12K	Summarize a requested paper within a concatenation of academic papers, ignoring the other documents.
choice_eng	base 95K	Select the correct multiple-choice answer to a question about a long English narrative.
many_shot	base 24K short 15K tiny 8K	Infer tasks from many demonstrations, including date understanding, translation-error detection, and shuffled-object tracking.
prefix_suffix	base 112K mid 56K short 19K tiny 8K	Recover a dictionary string matching both a supplied prefix and suffix, despite partial-match distractors.
vt	base 125K	Track variable assignments through a noisy context and identify variables sharing a requested value.
summary_with_needles	base 105K	Use one context for both document summarization and retrieval of inserted passkeys.
repoqa_and_kv	base 70K	Alternate semantic function retrieval and exact key-value lookup within a shared repository context.

D RETENTION–SCORE CURVES

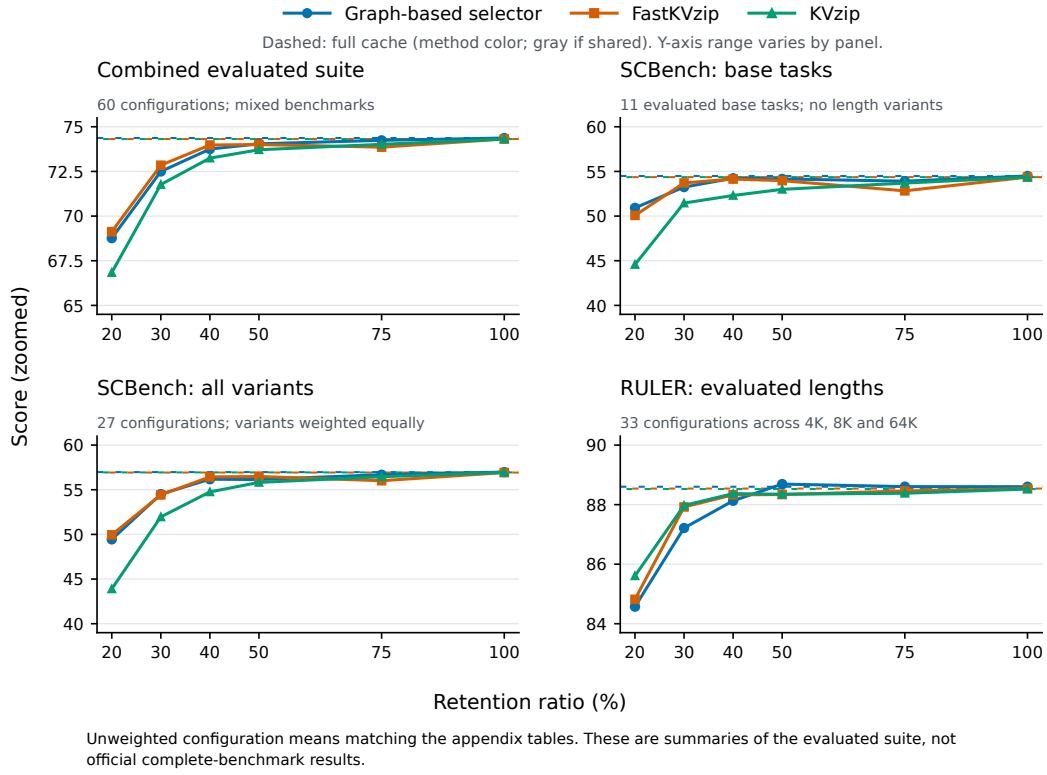
This appendix plots all 60 matched configurations and their benchmark and task-family summaries. The seven benchmark summaries reproduce the aggregate rows in Tables 1–3. Each aggregate is the arithmetic mean of its unrounded configuration scores, independently at each retention ratio. All plots use absolute native-evaluator scores (0–100) and the same method colors. Each panel has a zoomed linear y-axis enclosing all three methods and their full-cache baselines. Y-axis ranges vary across panels; tick labels show the actual scores. Flat and near-flat panels span at least two score points, and identical constant curves are explicitly marked.

Solid curves show the three methods at 20, 30, 40, 50, 75, and 100% retention. Dashed horizontal lines show each method’s own measured full-cache score, using the matching color; one gray dashed line is used when all three full-cache scores coincide. Coincident curves and nearly equal baselines may overlap. The full-cache point is included at 100% retention. SCBench base-task and all-variant means are distinguished; RULER 64K contains only seven selected tasks.

SCBench families follow Section 3.1 and Table 2 of Li et al. (2025b): string retrieval, semantic retrieval, global information processing, and multi-tasking. Chinese QA is absent from our semantic-retrieval subset. Prepared length variants inherit their base task’s family. RULER families and needle-retrieval subtypes follow Section 3 of Hsieh et al. (2024). One-task families repeat their individual curve for convenient comparison. The deployed combined scores for mixed SCBench and Many-shot tasks are retained; component scores are not separately available in the table snapshot. Unevaluated configurations have no curves.

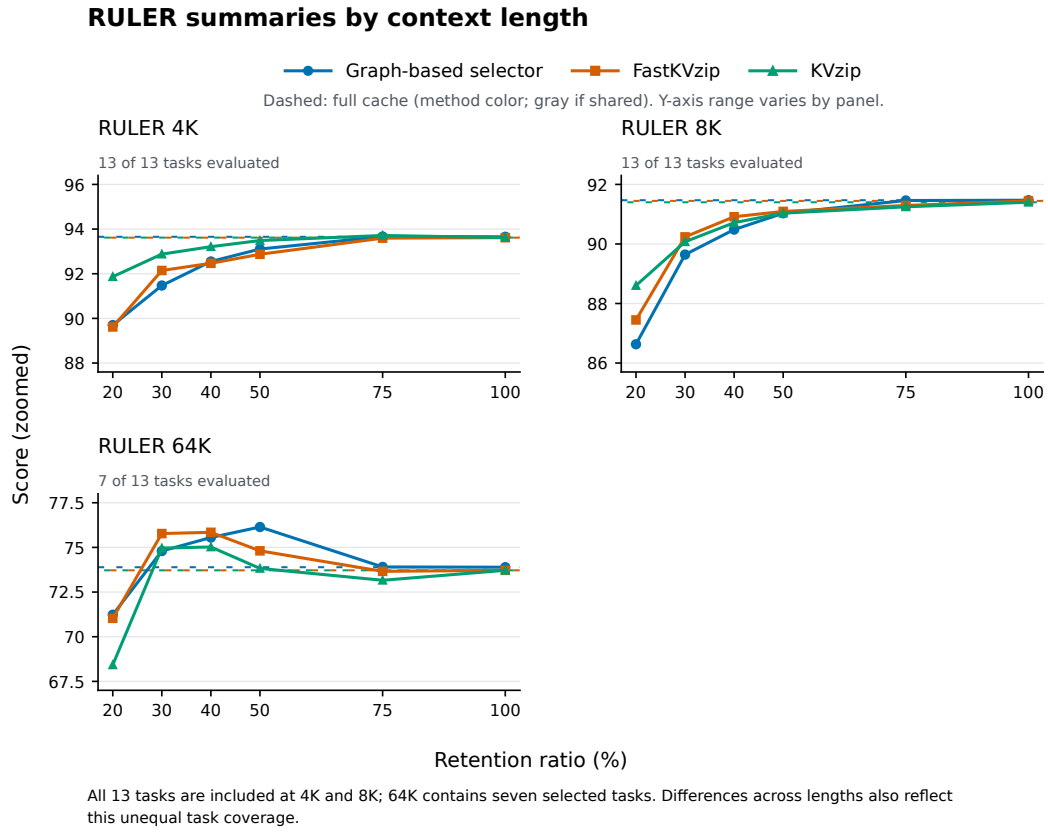
Plot guide. Benchmark summaries (Figures 3–4); SCBench task families (Figures 5–6); RULER task families (Figures 7–9); RULER retrieval subtypes (Figures 10–12); SCBench individual tasks (Figures 13–17); RULER 4K individual tasks (Figures 18–20); RULER 8K individual tasks (Figures 21–23); RULER 64K individual tasks (Figures 24–25).

D.1 BENCHMARK SUMMARIES

Benchmark summaries

01 / 23

Figure 3: Benchmark summaries.

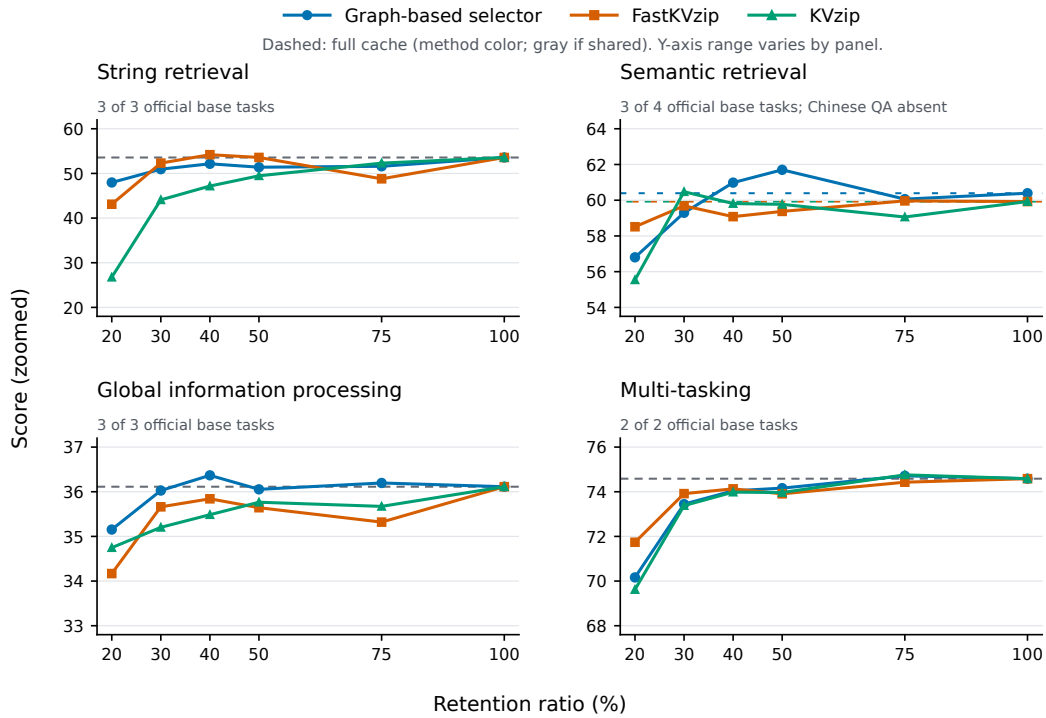


02 / 23

Figure 4: RULER summaries by context length.

D.2 SCBENCH TASK FAMILIES

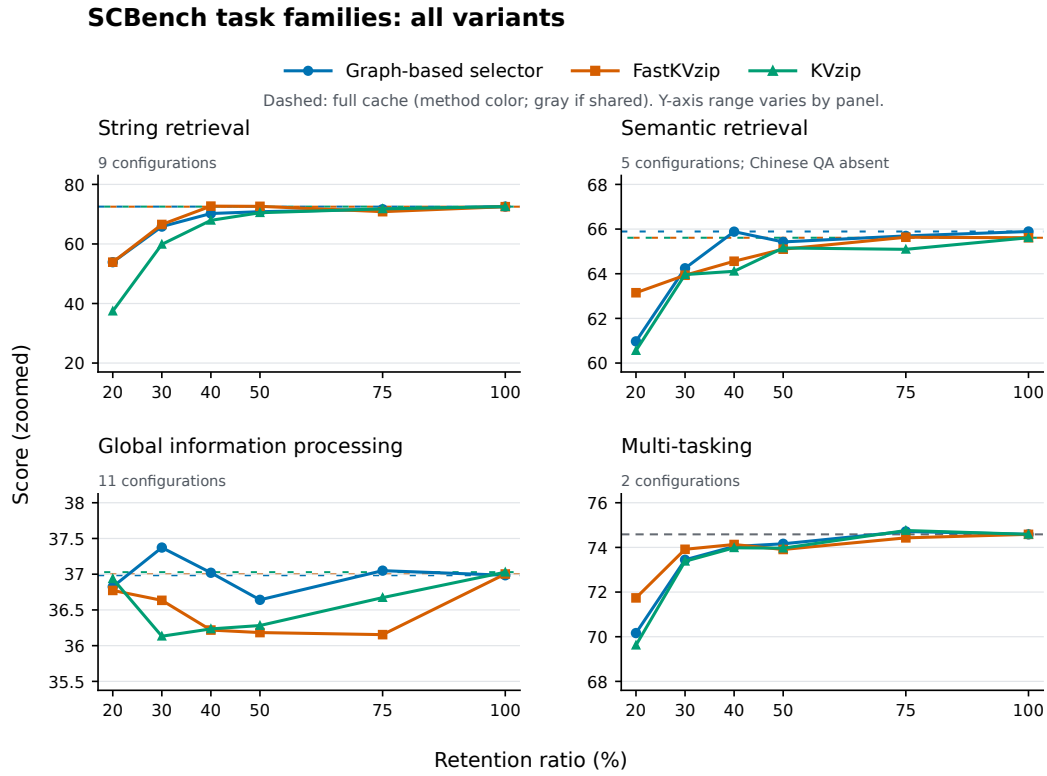
SCBench task families: base tasks



Categories follow SCBench Section 3.1 and Table 2. Semantic retrieval excludes the unevaluated Chinese-QA task. Only the 11 evaluated base tasks enter these means; prepared length variants are excluded.

03 / 23

Figure 5: SCBench task families: base tasks.

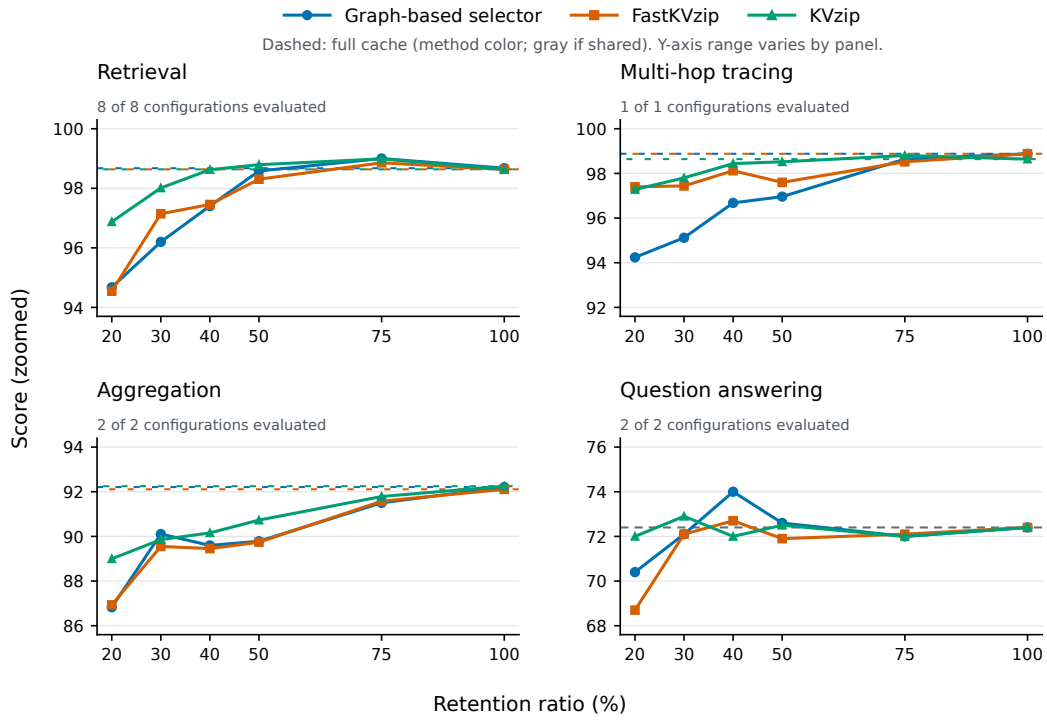


Categories follow SCBench Section 3.1 and Table 2. Semantic retrieval excludes the unevaluated Chinese-QA task. Prepared length variants inherit their base task's category and receive equal weight.

04 / 23

Figure 6: SCBench task families: all variants.

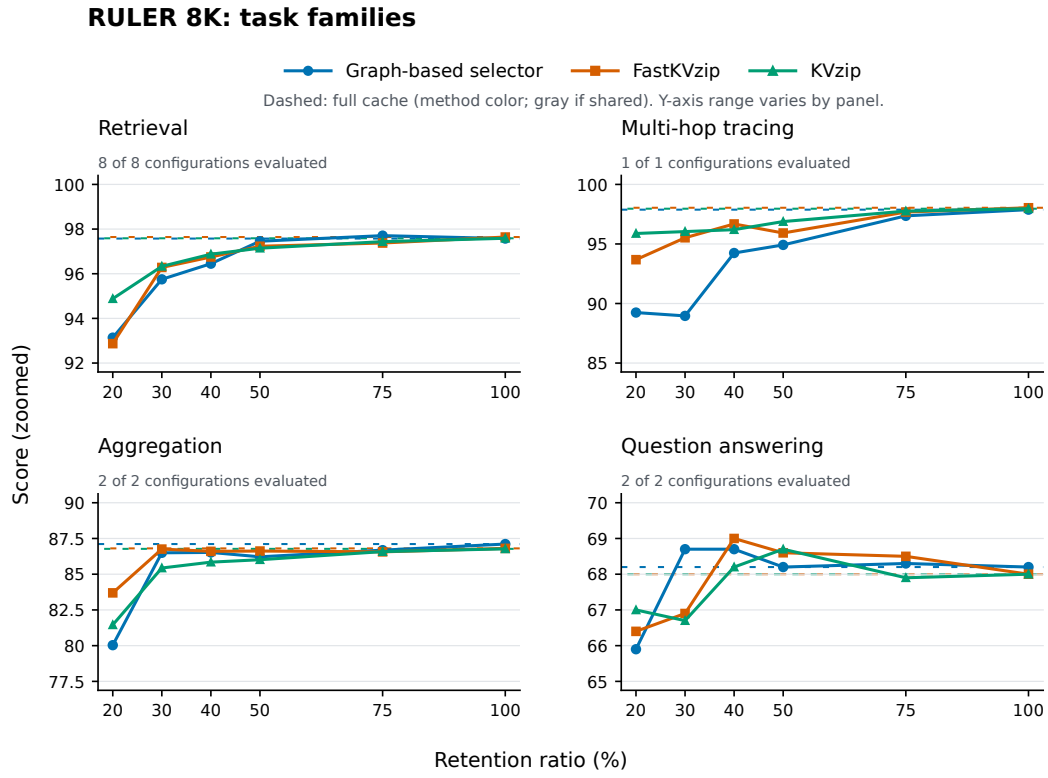
D.3 RULER TASK FAMILIES

RULER 4K: task families

Categories follow RULER Section 3. A one-configuration family reproduces its individual task curve.

05 / 23

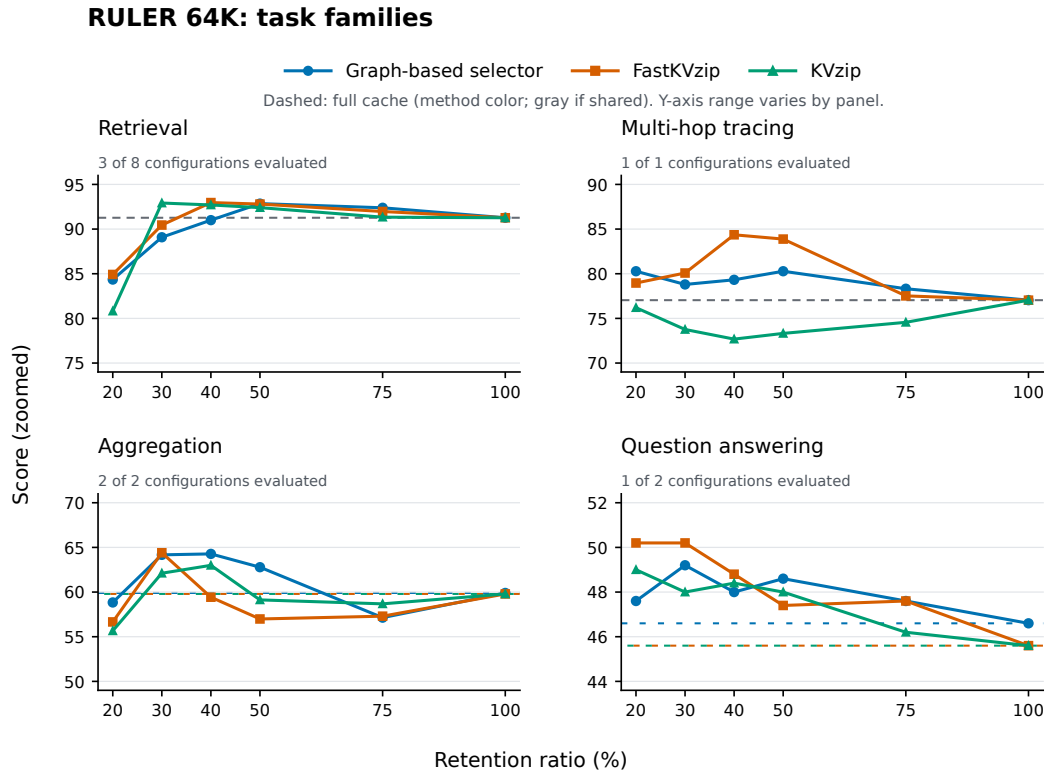
Figure 7: RULER 4K: task families.



Categories follow RULER Section 3. A one-configuration family reproduces its individual task curve.

06 / 23

Figure 8: RULER 8K: task families.

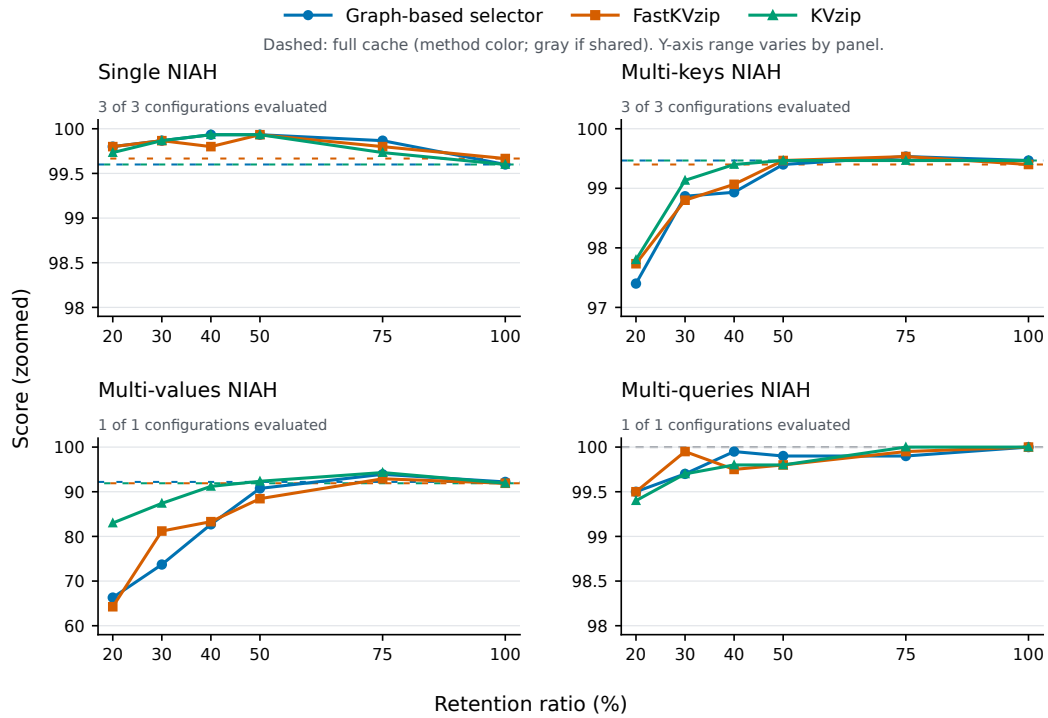


Categories follow RULER Section 3. Only the seven evaluated 64K tasks are included. A one-configuration family reproduces its individual task curve.

07 / 23

Figure 9: RULER 64K: task families.

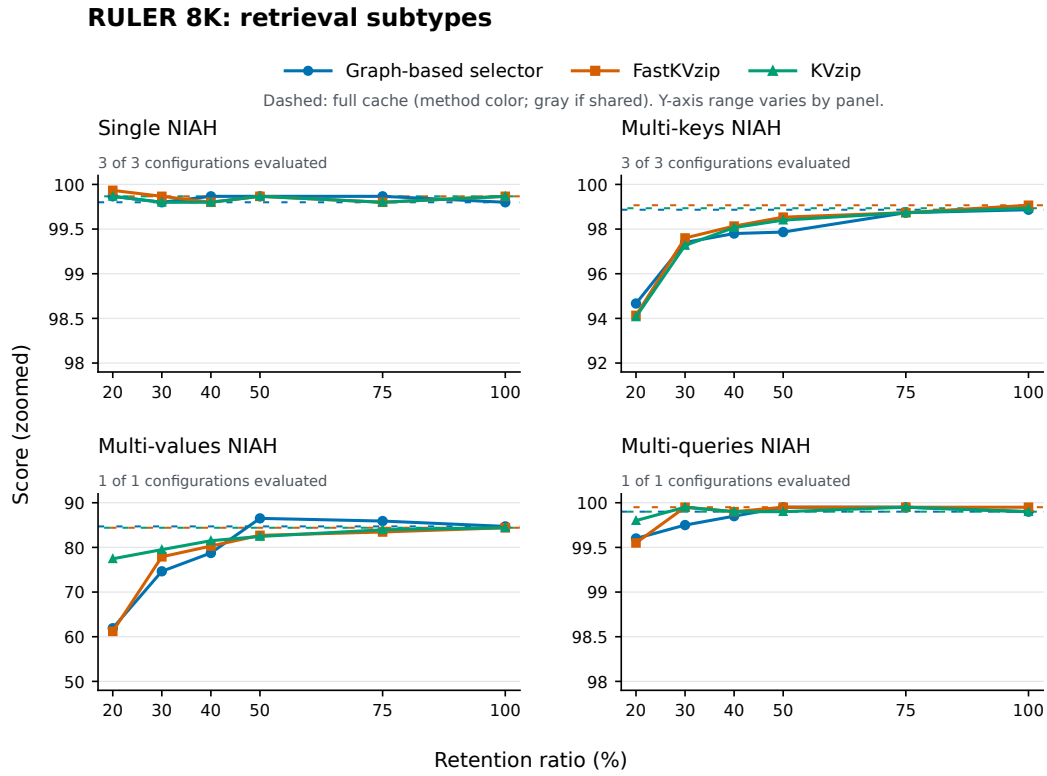
D.4 RULER RETRIEVAL SUBTYPES

RULER 4K: retrieval subtypes

Categories follow RULER Section 3. Single and multi-key means combine their available variants. A one-configuration family reproduces its individual task curve.

08 / 23

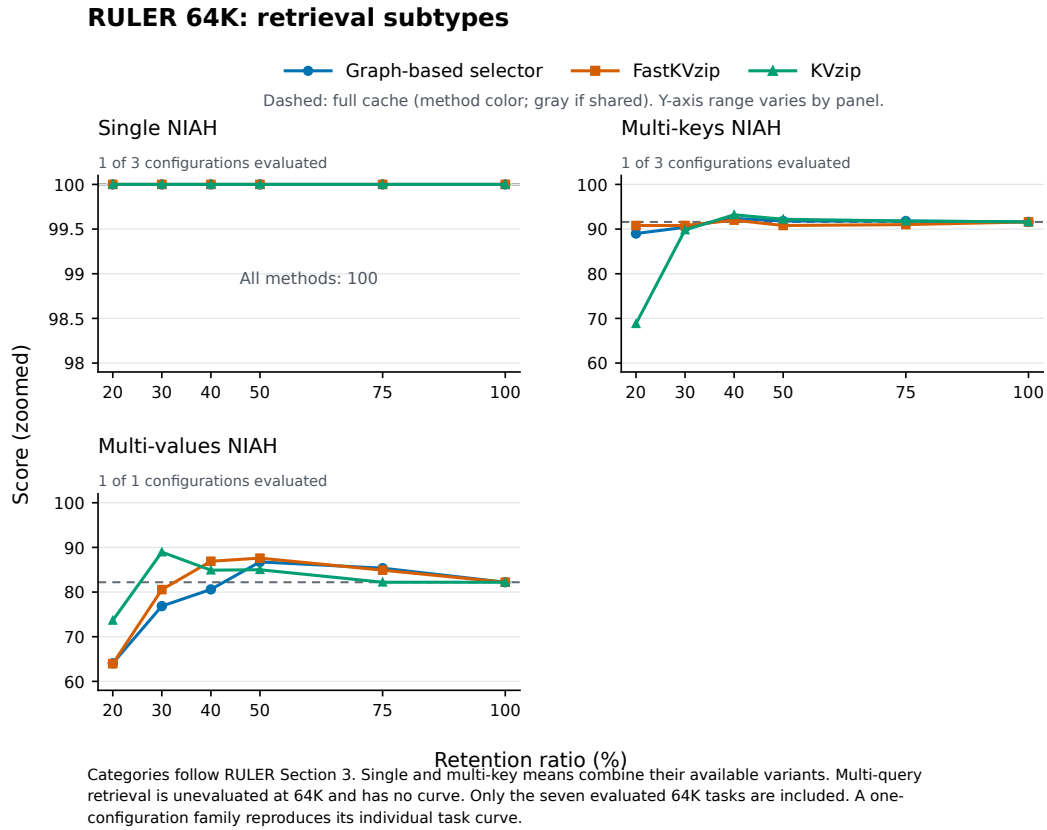
Figure 10: RULER 4K: retrieval subtypes.



Categories follow RULER Section 3. Single and multi-key means combine their available variants. A one-configuration family reproduces its individual task curve.

09 / 23

Figure 11: RULER 8K: retrieval subtypes.

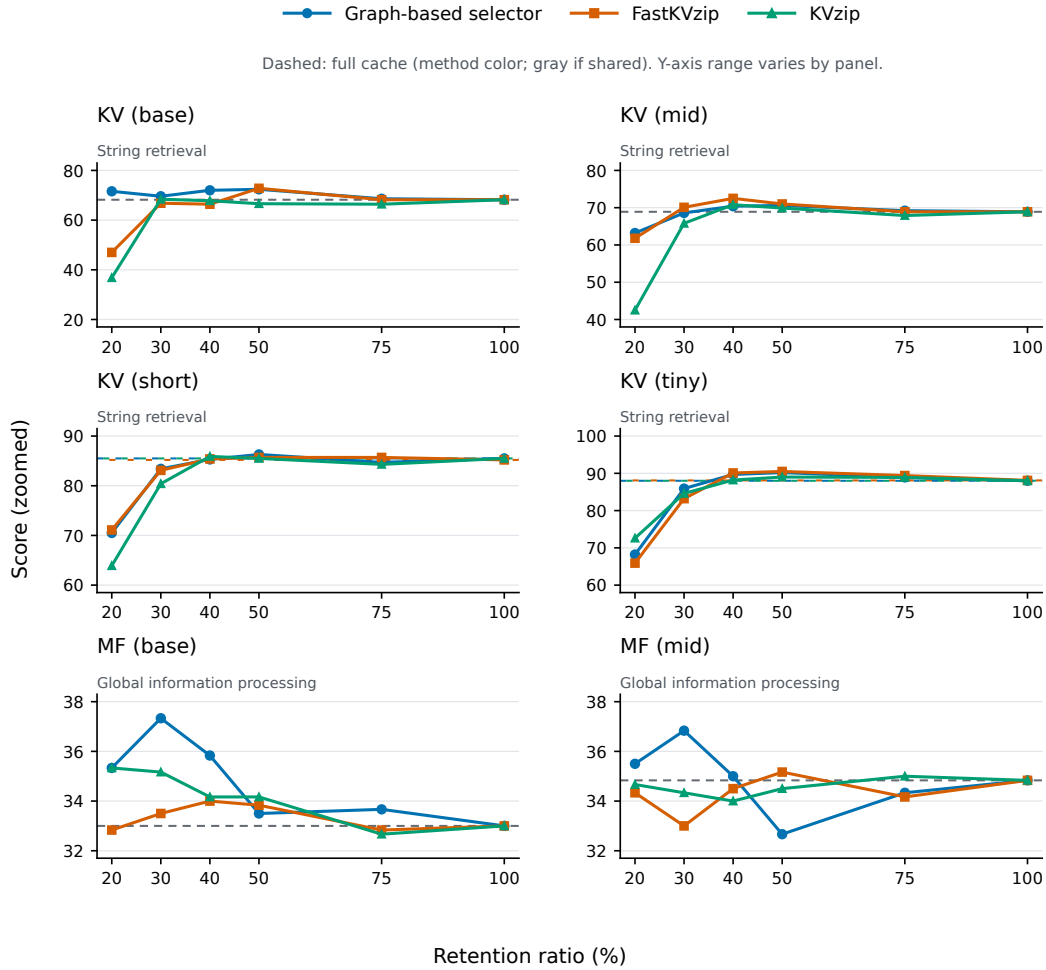


10 / 23

Figure 12: RULER 64K: retrieval subtypes.

D.5 SCBENCH INDIVIDUAL TASKS

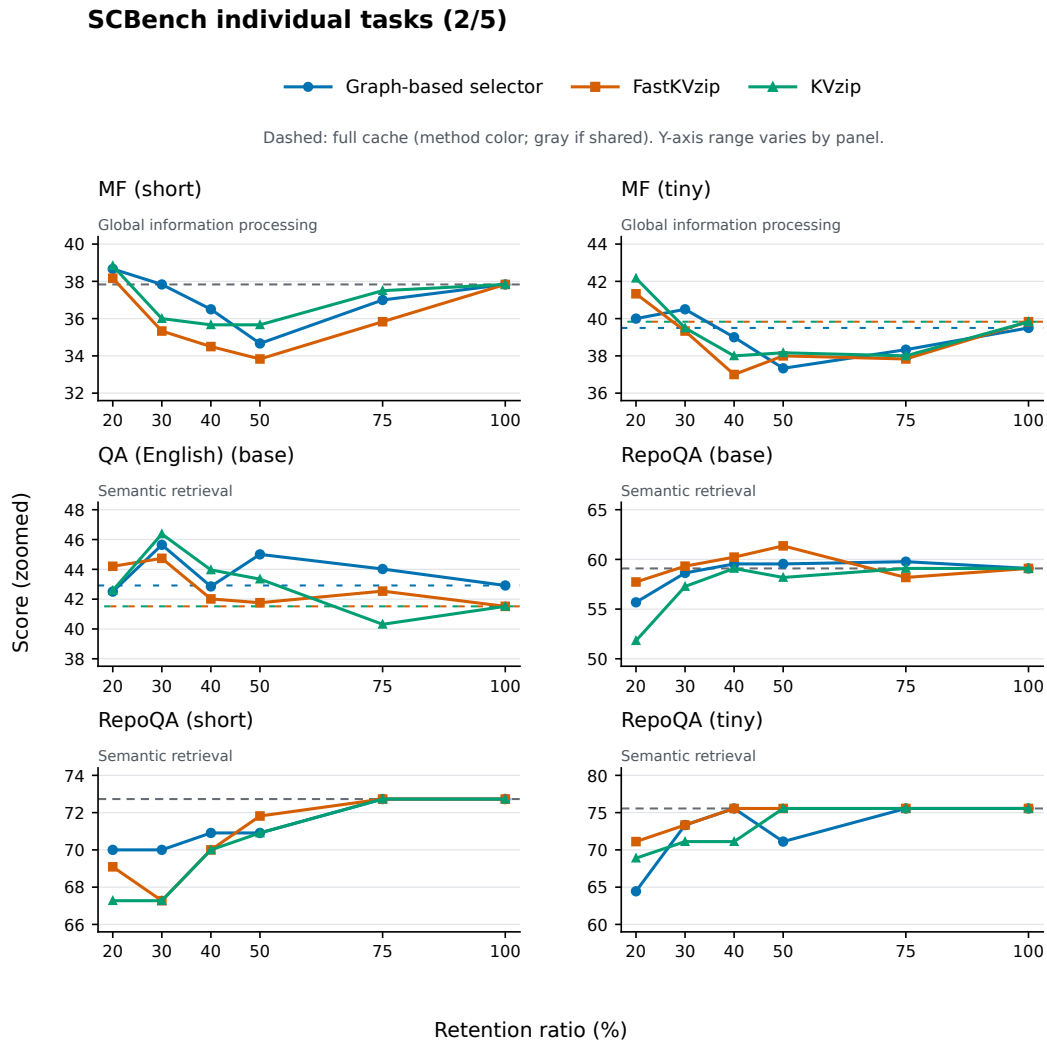
SCBench individual tasks (1/5)



Each panel is one evaluated configuration. Base, mid, short and tiny name the prepared variants used in the tables; mixed tasks retain their deployed combined score.

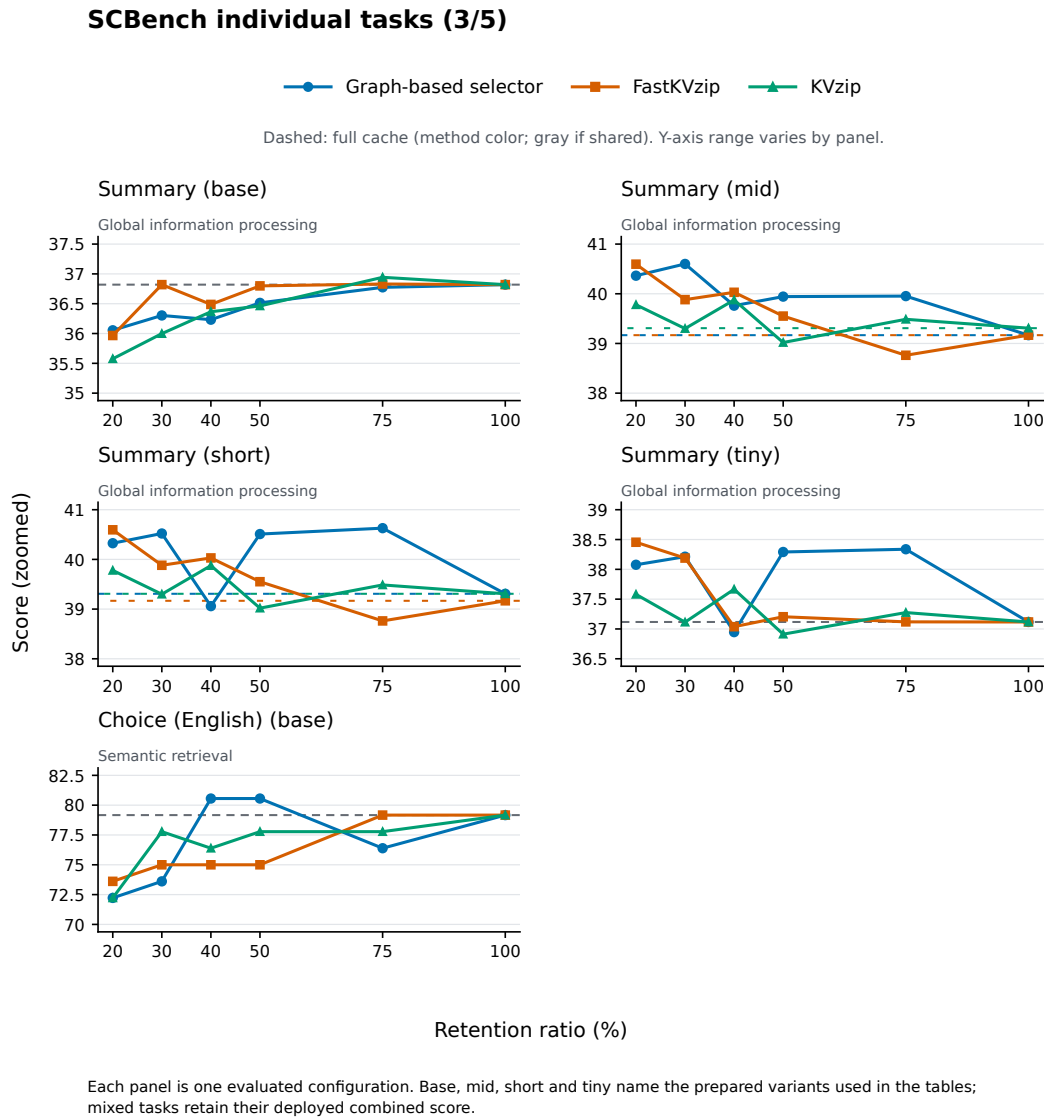
11 / 23

Figure 13: SCBench individual tasks (1/5).



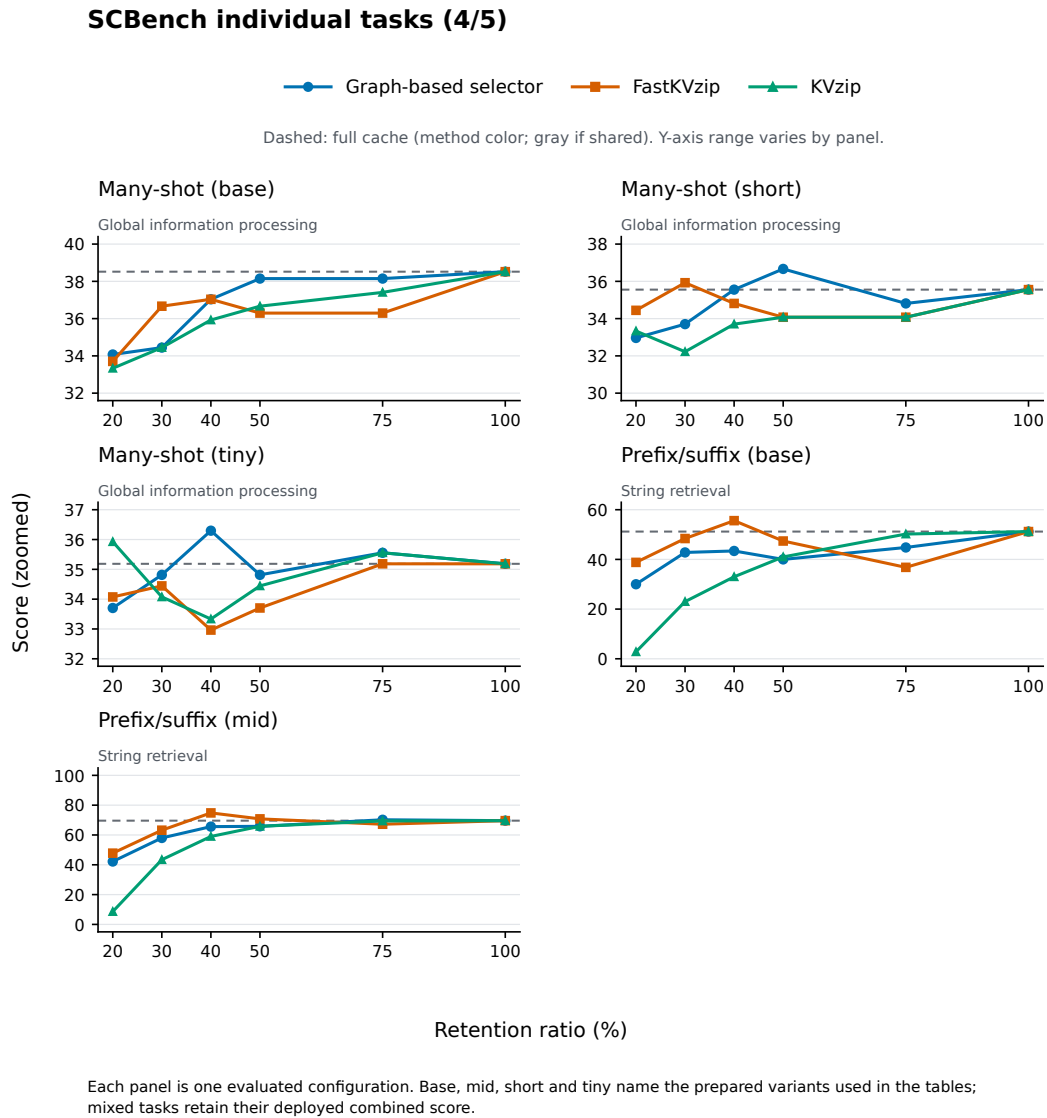
12 / 23

Figure 14: SCBench individual tasks (2/5).



13 / 23

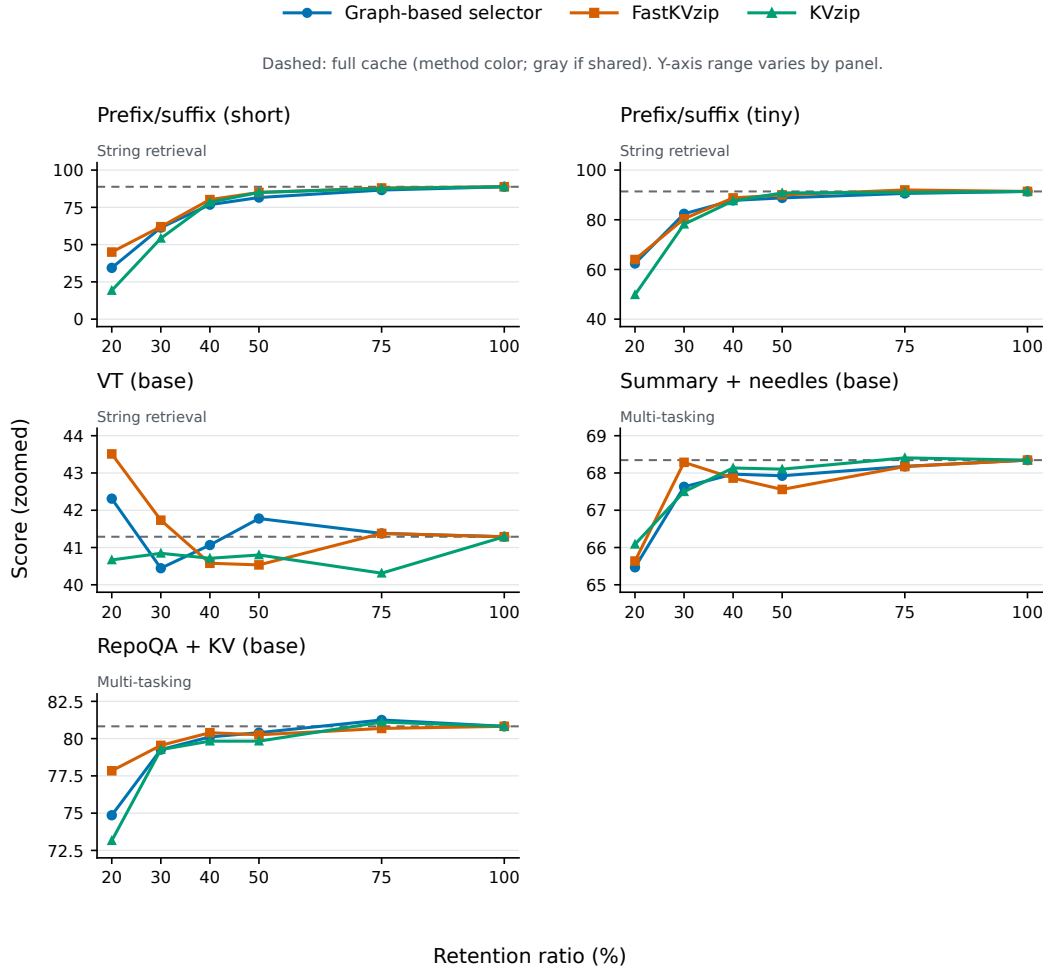
Figure 15: SCBench individual tasks (3/5).



14 / 23

Figure 16: SCBench individual tasks (4/5).

SCBench individual tasks (5/5)



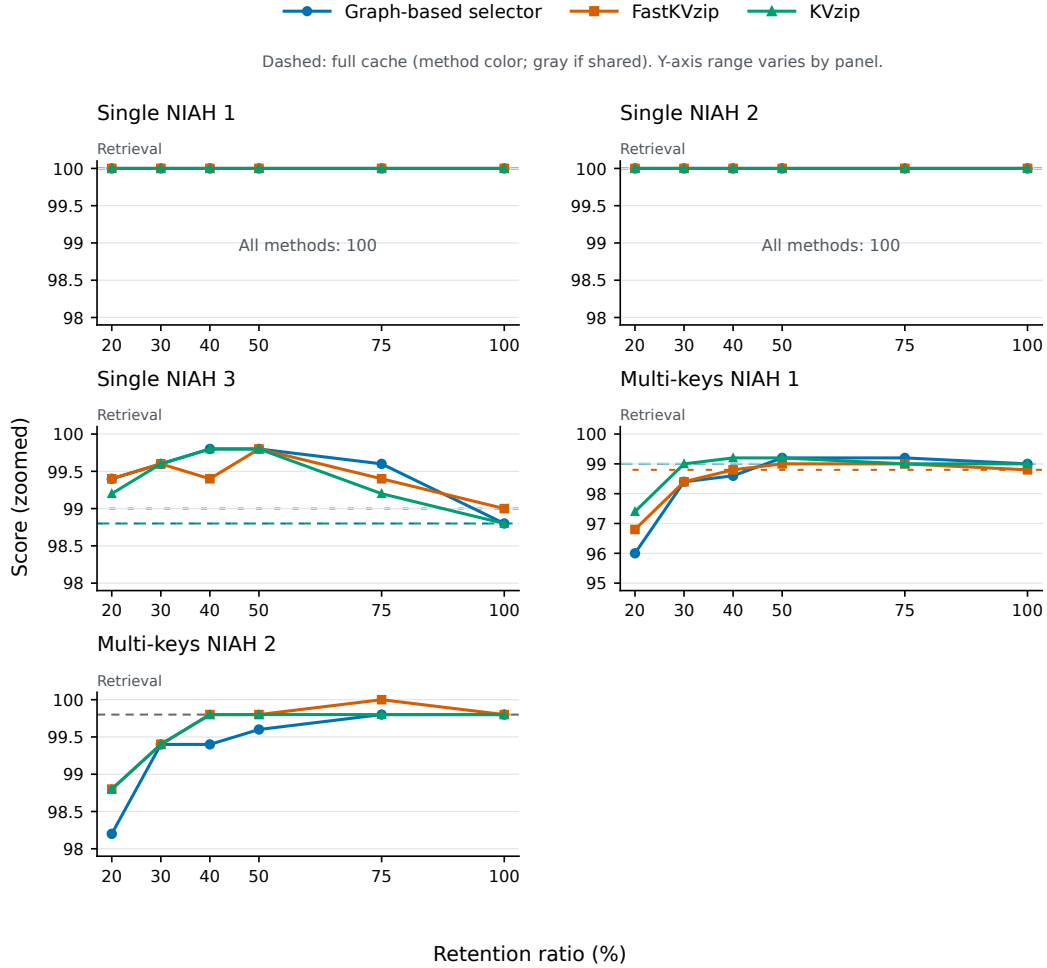
Each panel is one evaluated configuration. Base, mid, short and tiny name the prepared variants used in the tables; mixed tasks retain their deployed combined score.

15 / 23

Figure 17: SCBench individual tasks (5/5).

D.6 RULER 4K INDIVIDUAL TASKS

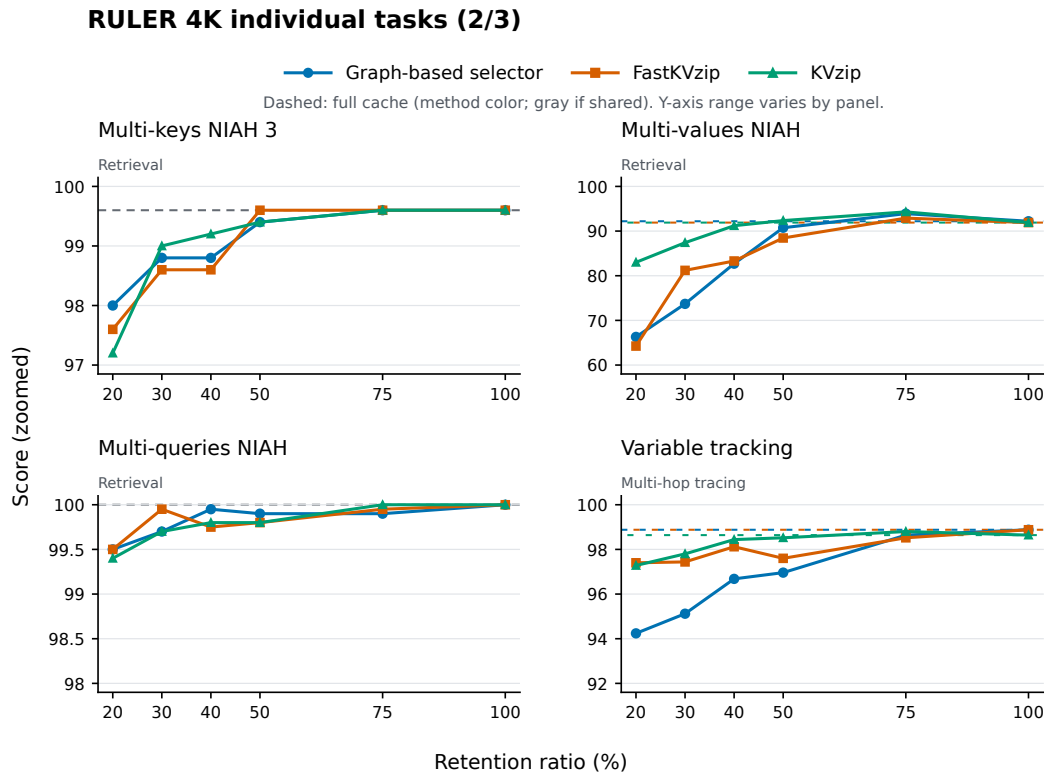
RULER 4K individual tasks (1/3)



Each panel is one evaluated 4K configuration. All 13 RULER task configurations are included across this section.

16 / 23

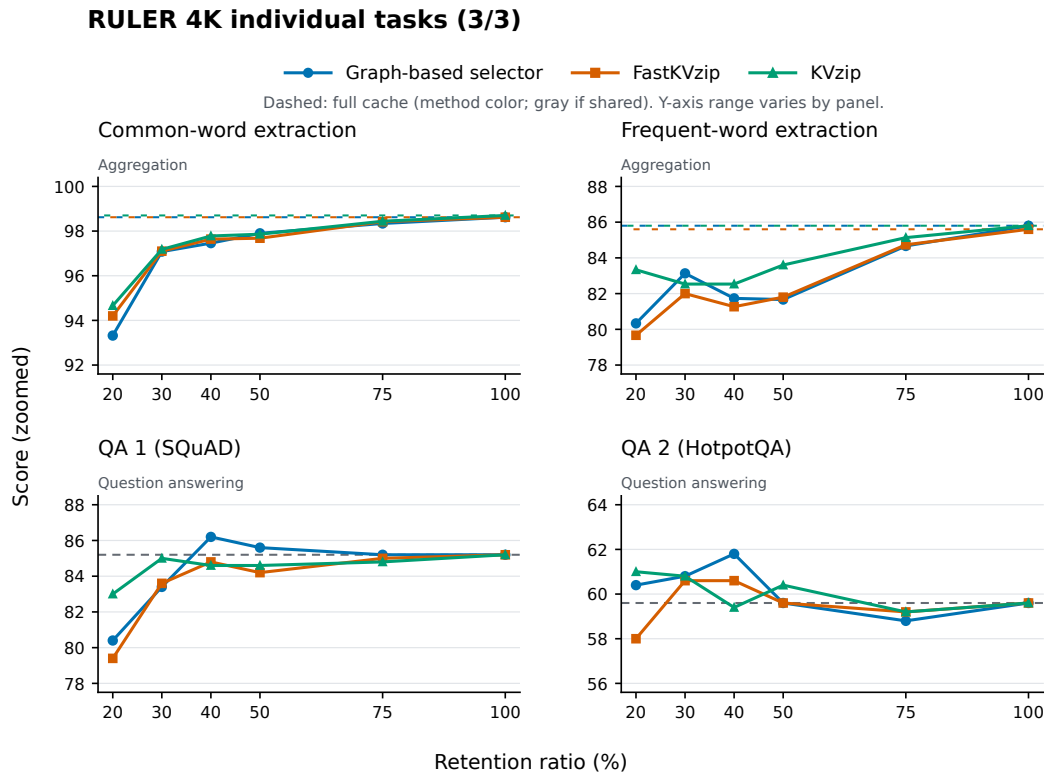
Figure 18: RULER 4K individual tasks (1/3).



Each panel is one evaluated 4K configuration. All 13 RULER task configurations are included across this section.

17 / 23

Figure 19: RULER 4K individual tasks (2/3).



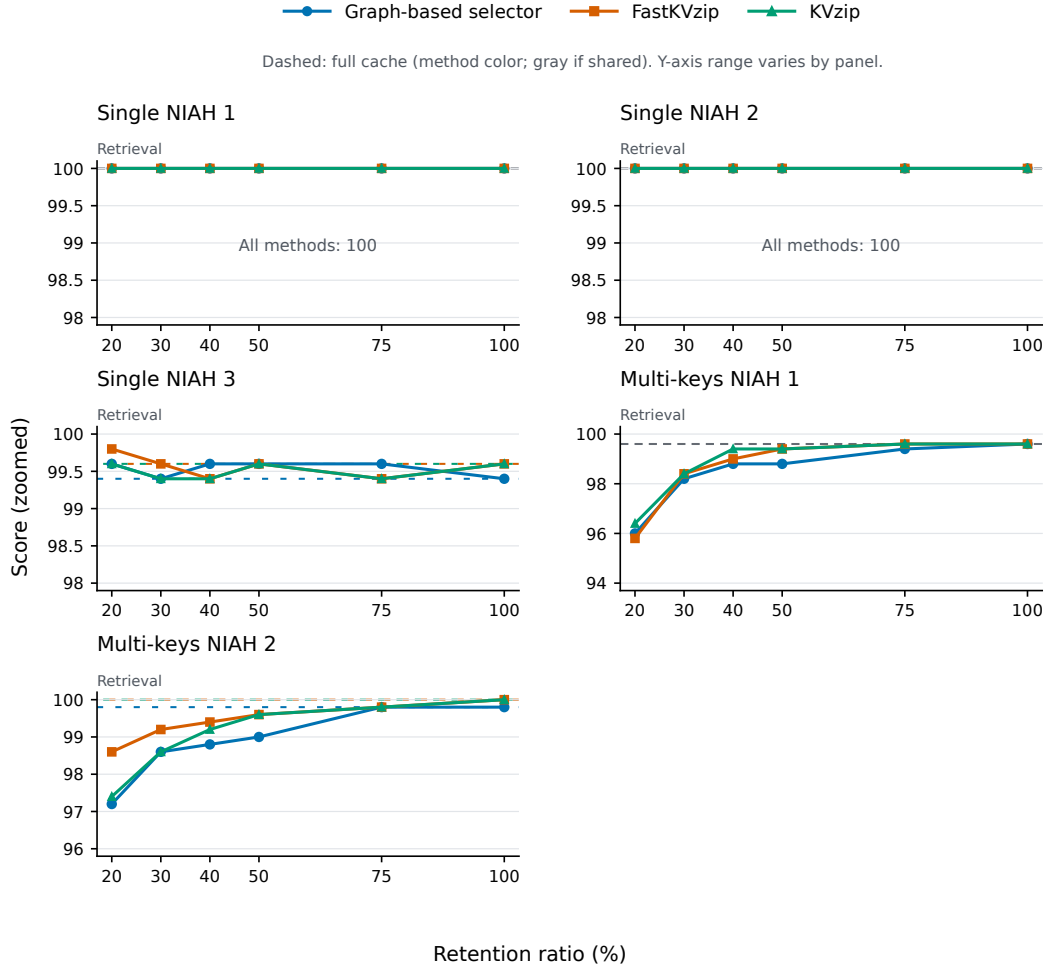
Each panel is one evaluated 4K configuration. All 13 RULER task configurations are included across this section.

18 / 23

Figure 20: RULER 4K individual tasks (3/3).

D.7 RULER 8K INDIVIDUAL TASKS

RULER 8K individual tasks (1/3)



Each panel is one evaluated 8K configuration. All 13 RULER task configurations are included across this section.

19 / 23

Figure 21: RULER 8K individual tasks (1/3).

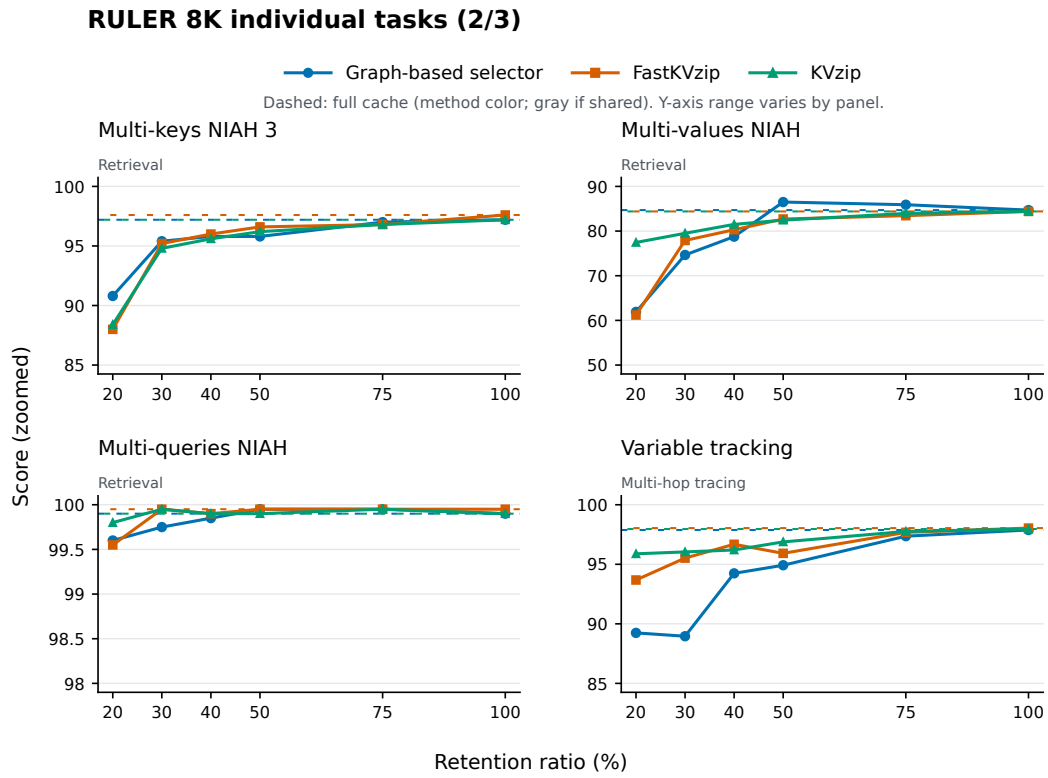
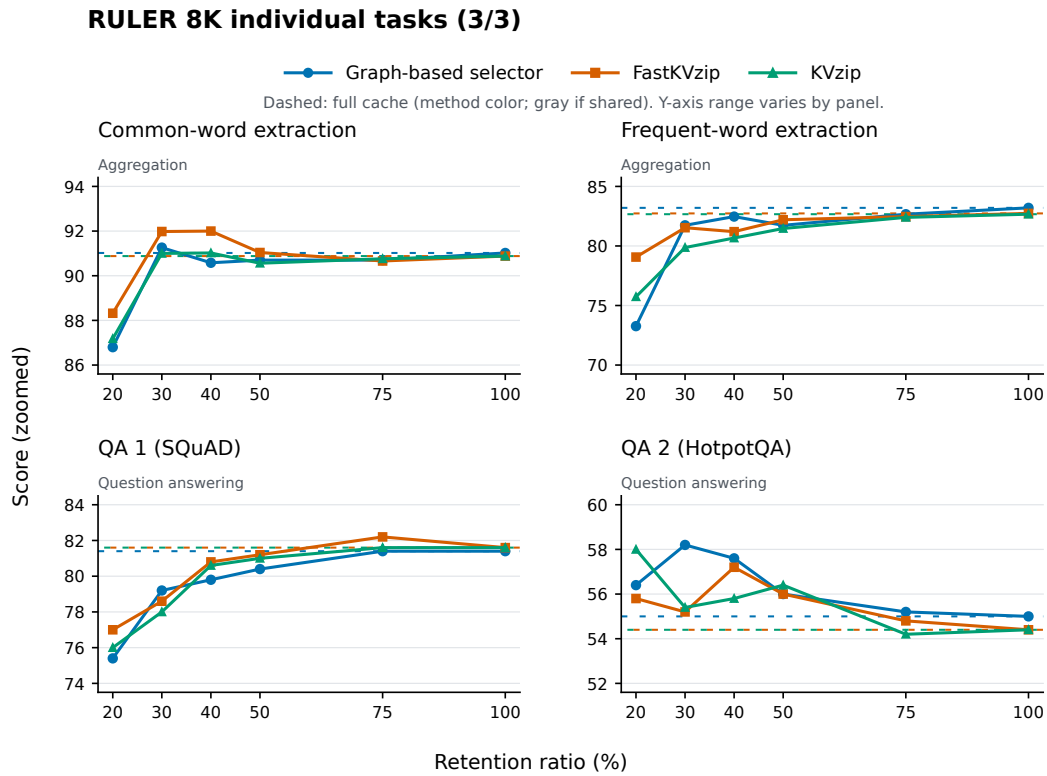


Figure 22: RULER 8K individual tasks (2/3).

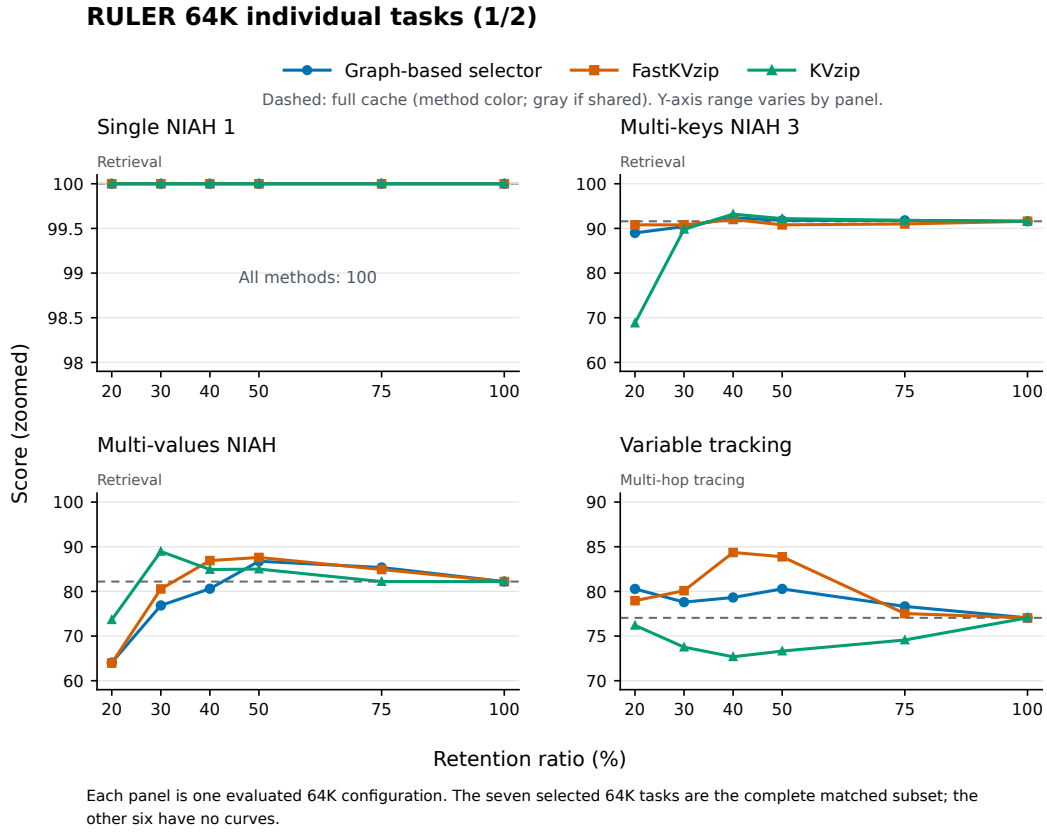


Each panel is one evaluated 8K configuration. All 13 RULER task configurations are included across this section.

21 / 23

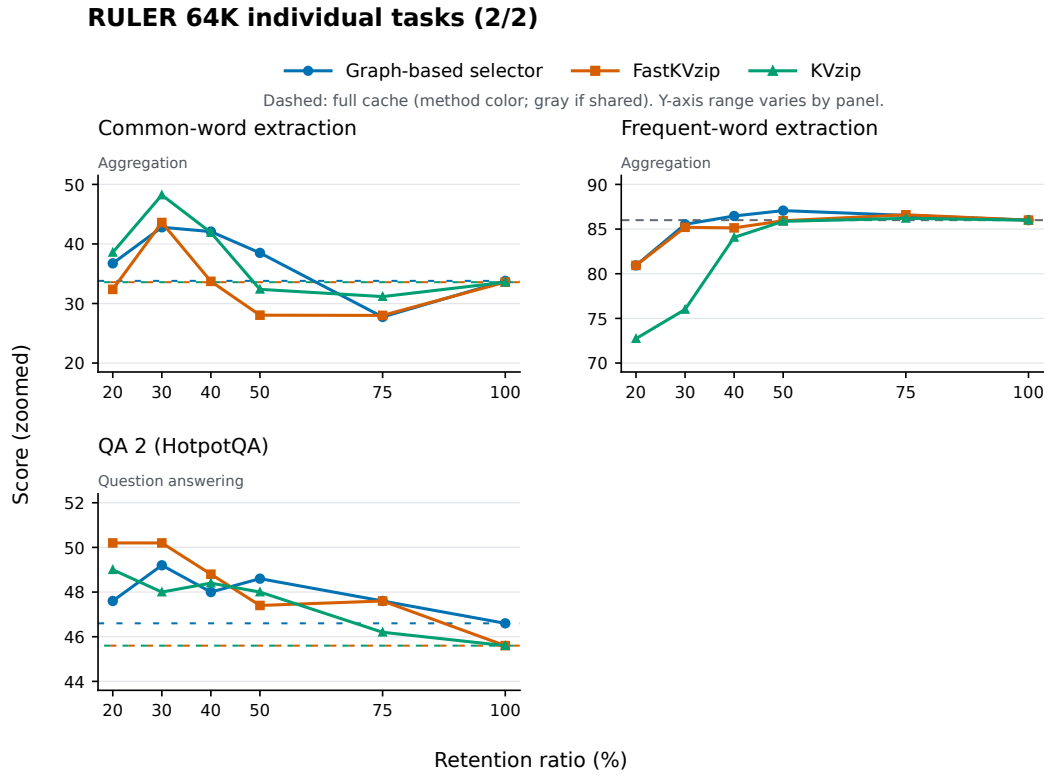
Figure 23: RULER 8K individual tasks (3/3).

D.8 RULER 64K INDIVIDUAL TASKS



22 / 23

Figure 24: RULER 64K individual tasks (1/2).



Each panel is one evaluated 64K configuration. The seven selected 64K tasks are the complete matched subset; the other six have no curves.

23 / 23

Figure 25: RULER 64K individual tasks (2/2).